ÇİP TASARIM YARIŞMASI
MİKRODENETLEYİCİ TASARIM KATEGORİSİ DETAY TASARIM RAPORU

TAKIM ADI: Kasırga - Ay

TAKIM ID: 919065

BAŞVURU ID: 4983815

İçindekiler

1. Sistem Tanımı ve Temel Tasarım Özeti 3
2. Proje Detay Tasarımı 4
    2.1. Sistem Mimarisi 4
    2.2. Tasarım Detayları 6
        2.2.1. İşlemci ve Bus Yapısının Tasarımı 6
        2.2.2. Çevre Birim Tasarım Detayları 7
        2.2.3. YZ Hızlandırıcı Tasarım Detayları 11
            2.2.3.1 TFLite Micro Speech Modeli 12
            2.2.3.2 RTL Tasarım Akışı 13
            2.2.3.3 YZ Hızlandırıcı Veri Akışı 18
        2.3. Boot 19
        2.4. FPGA Prototipleme 20
3. Çip Tasarım Akışı 21
4. Test 24
5. Takım Organizasyonu 29
    5.1. Takım Tanımı 29
    5.2. Görev Dağılımı 30
6. İş Planı ve Risk Planlaması 30
7. Kaynakça 32

1. Sistem Tanımı ve Temel Tasarım Özeti

Bu raporda Teknofest 2026 Çip Tasarım Yarışması, Mikrodenetleyici Tasarım Kategorisi için Kasırga - Ay takımı tarafından geliştirilen yapay zeka hızlandırıcı destekli mikrodenetleyicinin detaylı tasarımı anlatılmaktadır. Yarışma sürecinde, şartnamede belirtilen isterlere uygun yüksek başarımlı bir mikrodenetleyicinin oluşturulması hedeflenmiştir.

Hazırlanan Detay Tasarım Raporunun içeriği şu şekildedir: Proje Detay Tasarımı başlığı altındaki ilk kısım olan Sistem Mimarisinde mikrodenetleyicinin sistem tasarımından bahsedilmiş ve sistem tasarımının yarışma şartnamesinde belirtilen isterlere nasıl uyduğu izah edilmiştir. Ek olarak, tasarımın blok şemasına da yer verilmiştir. Tasarım Detayları alt başlığında ise işlemci ve veri yolu mimarisine, çevre birimlerinin her birine ayrı ayrı kendi başlıkları altında ve yapay zeka hızlandırıcının detaylarına değinilmiştir. Bu alt başlıkta alınan bütün tasarım kararları da yine ilgili kısımlarda açıklanmıştır. Önyükleme mekanizmasına ve FPGA prototiplemeye ise kendi alt başlıklarında değinilmiştir. Raporun Çip Tasarım Akışı kısmında fiziksel tasarımdan ve bu süreçte gerçekleştirilmiş olan çalışmalardan bahsedilmiştir. 3. başlık olan Test başlığı altında gerçekleştirilmiş olan ve gelecekte de yapılması planlanan test senaryoları kapsamlı bir planın parçaları olarak aktarılmıştır. Takım Organizasyonu başlığı altında takım üyeleri tanıtılmış, hangi üyeye hangi görevin düştüğü belirtilmiş ve takım hiyerarşisi verilmiştir. İş Planı ve Risk Planlaması kısmında ise projenin zaman planlaması hakkında detaylı bilgiler verilmiş ve iş paketlerinin ne kadarının tamamlandığına değinilmiştir. Son olarak Kaynakça başlığı altında tasarım sürecinde şimdiye kadar yararlanılmış olan ve bu raporda bahsedilen tüm kaynaklara yer verilmiştir.

Tasarlanan mikrodenetleyicide OpenHW Group tarafından sunulan CV32E40P [1] çekirdeği kullanılmıştır. Çekirdek 4 aşamalı boru hattı (*-ing.* pipeline) ile çalışmaktadır ve işlemler sırasında sıralı yürütüm (*-ing.* in-order execution) gerçekleşmektedir. Şartname uyarınca tasarlanması istenilen çevre birimlerinin tamamı takımımız tarafından tasarlanmıştır. Timer, UART, I2C Master, QSPI Master ve GPIO çevre birimlerinin RTL tasarımları tamamlanmıştır. CV32E40P işlemci çekirdeği sisteme entegre edilmiştir. Bunların yanında bütün çevre birimleri AXI4-Lite arayüzüne sahiptir ve AXI4-Lite arayüzü ile entegrasyonları tamamlanmıştır. Ayrıca OBI-to-AXI köprüsünün tasarımı tamamlanmış ve bu sayede haberleşmeyi OBI (Open Bus Interface) protokolü aracılığıyla sağlayan işlemcimiz ile şartnamede belirtildiği gibi AXI bağlantısı üzerinden haberleşilebilir hâle gelinmiştir. Projenin Github Repository'sine kaynakçada belirttiğimiz bağlantıdan ulaşılabilmektedir [2]. Çevre birimlerinden QSPI ve UART'ın çekirdeğe entegrasyonları ve testleri tamamlanmış kalan çevre birimlerinin entegrasyonu yapılmış ancak testleri henüz gerçekleştirilmemiştir.

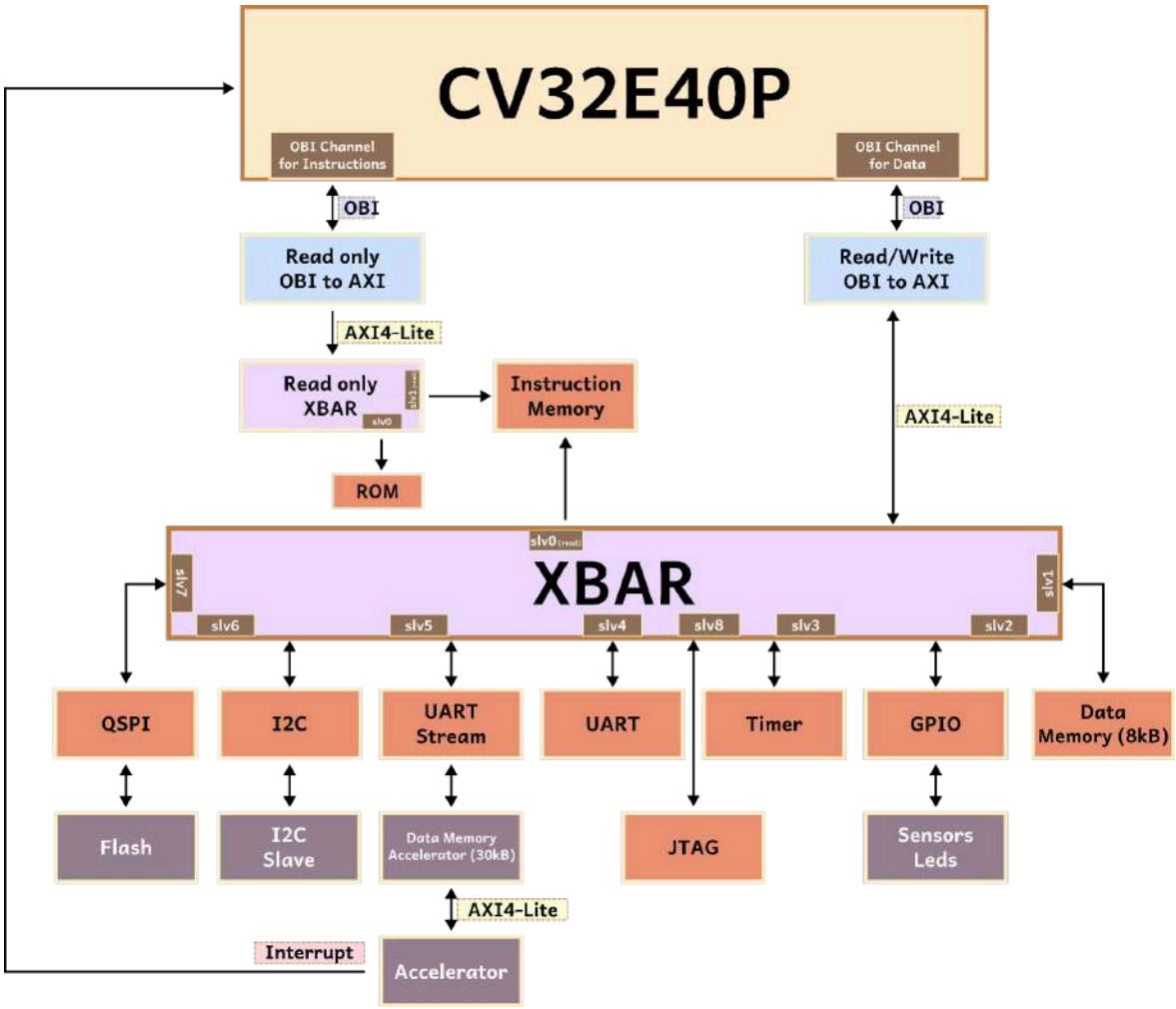
İşlemci çekirdeğinin doğrulanması için Spike [3] kullanılarak karşılaştırmalı testler yapılmıştır. Spike test modülü [4] yazılarak çekirdek ile bellek dosyası bağlanmıştır. Doğrulama için işlemcinin temel buyruklarını test eden bir C kodu yazılmış ve bu kod derlenerek bellek dosyasına yazılmıştır. Spike test modülünün içinde çekirdeğin işlediği buyruklar kaydedilmiştir. Spike simülatöründen alınan buyruklarla Spike test modülünden alınan buyruklar Python betikleri (*-ing.* script) kullanılarak otomatik olarak karşılaştırılmıştır.

Çevre birimleri doğrulanırken Vivado [5] üzerinden simülasyonlar yapılmış ve ayrıca cocotb [6] testlerine tabi tutulmuşlardır. Yapılan bu sınamalar sonucunda tasarımların beklenildiği gibi çalıştığı doğrulanmıştır. Ayrıca UART modülü Basys3 [7] FPGA (Field Programmable Gate Array) kartı aracılığıyla test edilmiş ve bu sayede donanım üzerinde de sağlaması gerçekleştirilmiştir. İlerleyen zamanlarda diğer çevre birimlerinin de fiziksel donanımda test edilmesi planlanmaktadır. Ayrıca fiziksel tasarım akışı için bir ön çalışma olması için UART modülü bütünüyle OpenLane [8] tasarım akışından geçirilmiştir [9].

YZ Hızlandırıcının RTL tasarımına ön değerlendirme raporunda planlanan zaman aralığına uygun olarak devam edilmektedir. Bu kapsamda yazılan mevcut modüller Vivado kullanılarak simülasyon üzerinde testbench testleri aracılığıyla test edilmiştir. RTL tasarımının tamamen bitmesinin ardından doğrulaması yapılacaktır ve FPGA üzerinde testlerden geçirilecektir.

2. Proje Detay Tasarımı

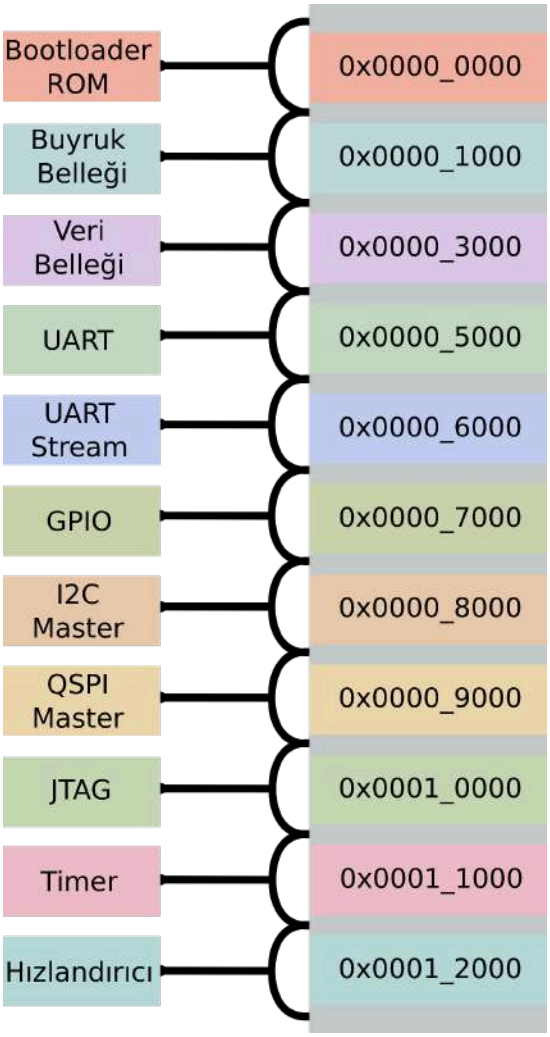
2.1. Sistem Mimarisi



Şekil 2.1.1: Sistem Mimari Diyagramı

Proje kapsamında tasarlanmış ve tasarım kararları yarışma şartnamesindeki isterlere binaen verilmiş olan mikrodenetleyicinin üst seviye Sistem Mimarisi Şekil 2.1.1'de verilmiştir. İşbu mimarinin merkezinde; 4 aşamalı boru hattına (*-ing*. pipeline) sahip, sıralı yürütüm (*-ing.* in-order execution) yapan ve 32-bit RV32IM[Zicsr][Zifencei]C buyruk kümesini destekleyen CV32E40P çekirdeği konumlandırılmıştır.

İşlemci OBI protokolünü kullanmaktadır. OBI, aktarımını el sıkışma (*-ing.* handshake) yöntemi ile yapar. İşbu protokol, şartname isterlerine göre tasarlanmış olan çevre birimlerinin kullandığı AXI4-Lite veri yolu protokolü ile tasarladığımız OBI-AXI köprüsü (*obi_to_axi.sv*) sayesinde rahatlıkla iletişim kurabilmektedir. AXI4-Lite ise AXI protokolünün, tüm işlemlerin uzunluğu 1 olan ve tüm veri erişimleri ve veri yolu genişliği aynı olan bir alt kümesidir. Bunların yanında AXI4-Lite bellek eşlemeli (*-ing.* memory mapped) erişimler için gerekli işlevselliği sağlar ve bunu AXI protokolüne göre daha az alan maliyeti ile yapar. Bu sayede güç tüketimi azalmakta ve sadeleştirilmiş bir alt küme olduğu için entegrasyonu da kolaylaşmaktadır.



AXI4-Lite slave arayüzüne sahip olan çevre birimlerinin her birinin Şekil 2.1.2'de verilen Bellek Haritasında da görüldüğü üzere kendilerine ait adresleri vardır. İşbu bellek haritasında, Ön Tasarım Raporu'nun yayınlanmasından sonra ekibimizce alınan karara binaen değişikliğe gidilmiştir, bu kararın sebebi olarak daha önce yapılmış olan bellek haritasındaki adres uyuşmazlığı problemi gösterilebilir. Tasarlamış olduğumuz XBAR (*AXI4-Lite crossbar*) modülü ile bellek haritasından istenilen slave hatlarına gerekli verileri göndermektedir. XBAR modülü ile ilgili raporun ilerleyen bölümlerinde detaylı bir açıklama mevcuttur. Şartnamede belirtildiği üzere çekirdek 8 kB buyruk (*-ing.* instruction) ve 8 kB veri (*-ing.* data) belleği içermektedir. Buna ek olarak YZ Hızlandırıcı için de 30 kB boyutunda bir bellek vardır. Ayrıca sistem ilk başlatıldığında çalıştırılan ve harici bir flash bellekte bulunan uygulama kodunu buyruk belleğine yükleyen ön yükleyici (*-ing*. bootloader) kodunun bulunduğu Bootloader Rom için de 1 kB boyutunda bir bellek vardır.

İşlemci üzerinde çalıştırılacak yazılımlar C dilinde geliştirilecektir. Bu kodlar riscv-gnu-toolchain [10] kullanılarak derlenecek ve oluşan dosyalar flash belleğe yüklenecektir. Ön yükleme işlemi ile beraber flash bellekteki bu kodlar okunarak buyruk belleğine yüklenir ve yürütme işlemi başlamış olur.

Şekil 2.1.2: Bellek Haritası

Mikrodenetleyici üzerinde program çalıştırılabilmesi için buyrukların ve statik verilerin doğru bellek adreslerine yerleştirilmesi gerekmektedir. Bu amaçla derleme aşamasında bağlayıcı betik (*-ing.* linker script) kullanılmıştır. Bağlayıcı betik, derlenen programın bellek haritasına göre doğru yerlere yerleştirilmesini sağlar.

Tasarımın doğrulanması amacıyla Basys3 FPGA kartı ile test süreci yürütülmüştür. Tasarımı tamamlanan donanım bileşenleri bir araya getirilerek tam sistem simülasyonları gerçekleştirilmiş, ardından tasarım FPGA üzerinde gerçeklenmiş ve donanım düzeyinde de testlere tabi tutulmuştur. FPGA prototiplemesi sürecinde Xilinx Vivado Design Suite kullanılmış ve tasarım sentezlenerek Basys3 FPGA kartına yüklenmiştir. UART çevre biriminin entegrasyonu FPGA üzerinde test edilmiş; haberleşmenin AXI4-Lite veri yolu ile sorunsuz şekilde gerçekleştiği ve yarışma isterlerine uygun çalıştığı doğrulanmıştır. Bu testler ile sistemin hem mimari bütünlüğü hem de donanım-yazılım uyumu başarıyla doğrulanmıştır.

FPGA üzerinde yapılan başarılı testlerin ardından, çipin alan kullanımı, güç tüketimi ve zamanlama davranışı gibi fiziksel özelliklerini değerlendirmek amacıyla ASIC gerçeklemesine yönelik çalışmalara başlanmıştır. Bu çalışmaların sonucunda sistemin gerçek üretim koşullarında nasıl davranacağının daha iyi anlaşılması hedeflenmektedir. Yarışma kapsamında kullanılacak ticari EDA araçlarına henüz erişim sağlanamadığı için öncü çalışmalar, açık kaynak fiziksel tasarım akışı aracı olan OpenLane kullanılarak gerçekleştirilmiştir. Bu kapsamda sistem alt modüllerinden olan UART çevre birimi; 180×180 µm^2 lik alanda, 20 nanosaniye saat vuruş sıklığına (50 MHz) sahip olacak şekilde fiziksel tasarım akışından geçirilmiştir. Tasarım, OpenLane çip tasarım akışının bütün aşamalarından başarılı bir şekilde geçmiştir. Bu çalışma ile birlikte, tasarımın ASIC akışına uygunluğu test edilmiş ve ilk gerçeklemesi başarı ile tamamlanmıştır. Ticari EDA araçlarına erişim sağlandığında daha kapsamlı entegrasyon çalışmaları, tasarımın daha detaylı doğrulanması ve iyileştirilmesi gerçekleştirilerek ASIC gerçeklemesinin nihai hale gelmesi sağlanacaktır. İşbu süreçte detaylı yerleştirme sonrası zamanlama analizi yapılacak, güç tüketimi ve kaynak kullanımı gibi çıktılar incelenecek performansın artırılması amacıyla fiziksel tasarım akışında çeşitli özelleştirmeler yapılacaktır.

2.2. Tasarım Detayları

Bu bölümde kullandığımız bus yapısından, çevre birimlerinin tasarımından ve takımız tarafından alınan tasarım kararlarında bahsedilmektedir.

2.2.1. İşlemci ve Bus Yapısının Tasarımı

Kullanmakta olduğumuz CV32E40P çekirdeği iki adet OBI arayüzüne sahiptir. Bu arayüzlerden bir tanesi sadece okuma özelliğine sahip olup işlemcinin buyruk getirmesi için kullanılan buyruk arayüzü, diğeri ise hem okuma hem de yazma özelliğine sahip olup veri okuma-yazma işlemlerinin gerçekleşmesini sağlayan veri arayüzüdür. İşlemcinin veri arayüzü, OBI master arayüzünü AXI4-Lite master arayüzüne dönüştüren bir modül kullanılarak (*obi_to_axi*) çevre birimleri ve crossbar ile iletişim kurabilir hâle getirilmiştir. OBI protokolünün AXI4-Lite ile benzer bir el sıkışma (*-ing.* handshake) protokolü kullanması, yalnızca okuma özelliğine sahip OBI arayüzünde modülün tamamen kombinasyonel olarak tasarlanabilmesini sağlamıştır. Bu sayede buyruk okuma işlemi iletişim sürecinde fazladan bir çevrim

maliyeti oluşturmadan sağlanabilmektedir. XBAR (*AXI4-Lite crossbar*) modülü bir veya daha fazla AXI4-Lite modülünün bir veya fazla sayıda AXI4-Lite slave modülü ile olan iletişimini düzenler. Bu yapı sayesinde, her master modülü istenilen adreslemeler çerçevesinde istediği slave modülü ile doğrudan iletişim kurabilir. Tasarımın güncel durumunda işlemci master, bellekler ve çevre birimleri slave olarak bağlanmıştır. XBAR modülü, okuma veya yazma buyruklarının istek yaptığı bellek adreslerini modüllerin bellek haritasında belirlenmiş olan adresleriyle karşılaştırarak AXI4-Lite sinyallerini doğru slave modülüne iletir. Bu sayede okuma ve yazma buyrukları doğrultusunda yazmaç değerleri işlemci tarafından güncellenebilir. Veri belleğinin yapay zeka hızlandırıcıyla doğrudan iletişim kurabilmesi için ikinci bir AXI4-Lite slave arayüzü vardır. Yapay zeka hızlandırıcı veri belleğinde yalnızca spesifik bir adres aralığına erişebildiği için veri belleğinin XBAR modülüne bağlı AXI4-Lite arayüzü ile çakışmaz.

İşlemcinin buyruk getirmek için kullanılan OBI arayüzü, OBI-AXI köprü bağlantısı ile XBAR modülüne master olarak bağlıdır. Ancak işlemcinin buyruk getirmek için kullanılan OBI arayüzü sadece okuma fonksiyonuna sahip olduğu için OBI-AXI köprüsünden sadece AXI4-Lite okuma kanalı kullanılır. XBAR modülüne buyruk belleği ve Boot ROM ise slave olarak bağlıdır ve yine sadece okuma sinyalleri yönlendirilir. Yani veri akışı işlemciye doğru tek yönlüdür. Ön yükleme aşamasında işlemcinin buyruk belleğine yazabilmesi için buyruk belleğinin bir AXI4-Lite slave modülü olarak XBAR modülüne bağlanması planlanmıştır. Yani buyruk belleğinin AXI4-Lite yazma arayüzü işlemcinin veri yazıp okumak için kullandığı arayüze bağlıdır. Okuma arayüzü ise işlemcinin buyruk getirmek için kullandığı arayüze bağlıdır. Buyruk belleğine sadece ön yükleme aşamasında veri yazılır. Ön yükleme sonrasında buyruk belleğine sadece çekirdek tarafından erişilir ve buyruk belleğine yapılan herhangi bir okuma isteği sistemi kilitlemez.

2.2.2. Çevre Birim Tasarım Detayları

Timer, UART, I2C, QSPI ve GPIO çevre birimlerinin RTL tasarımları ekibimizce tamamlanmış Vivado üzerinden Verilog testleri gerçekleştirilmiştir. Bunun yanında bütün çevre birimlerine AXI4-Lite arayüzü entegre edilmiş ve AXI4-Lite entegre Vivado Verilog testleri tamamlanmıştır. Çevre birimlerinden QSPI ve UART'ın çekirdeğe entegrasyonları ve testleri tamamlanmış kalan çevre birimlerinin entegrasyonu yapılmış ancak çekirdek entegre testleri henüz gerçekleştirilmemiştir. Çevre birimleri üzerinde çalışmalar devam etmektedir.

GPIO

GPIO (*-ing* General Purpose Input/Output) modülü; mikrodenetleyicinin sensörler, butonlar, anahtarlar, motorlar ve LED'ler gibi harici çevre birimleriyle dijital olarak iletişim kurmasını sağlayan arayüzdür. Modül üzerinde 16 adet giriş ve 16 adet çıkış olmak üzere toplam 32 adet harici pin bulunur. Bu pinlerin yönetimi, sisteme entegre edilmiş bir AXI4-Lite Slave arayüzü üzerinden GPIO_IDR ve GPIO_ODR yazmaçları vasıtasıyla gerçekleştirilir.

GPIO_IDR (*-ing* Input Data Register), giriş pinlerinin anlık mantık seviyelerini okumak için kullanılır. İşlemci okuma isteği gönderdiğinde pin durumları yazmacın alt 16 bitine (GPIO_IDR[15:0]) yansıtılır, üst 16 bit (GPIO_IDR[31:16]) ise her zaman sıfır olarak döner.

GPIO_ODR (*-ing* Output Data Register), çıkış pinlerini sürmek için kullanılır. Yazmacın alt 16 bitine (GPIO_ODR[15:0]) yazılan değerler doğrudan çıkış pinlerine yansıtılır. Üst 16 bitin ise (GPIO_ODR[31:16]) fiziksel çıkış pinleri üzerinde herhangi bir etkisi yoktur.

GPIO_IDR yazmacı yalnızca okunabilir (*-ing* read-only) olup yazma işlemleri etkisizdir. GPIO_ODR ise hem okunabilir hem yazılabilir (*-ing* read-write) olup okunduğunda mevcut çıkış durumunu döner.

GPIO modülümüz AXI4-Lite slave arayüzüne sahiptir.

Timer

Timer (sayaç) modülü; gecikme oluşturma, zaman ölçme, periyodik olayların takibini yapma gibi işlemlerin gerçekleştirilmesini sağlayan modüldür. Bu modül sisteme entegre edilmiş olup testleri devam etmektedir. Timer modülünün test edilmesi için gerekli C ve header dosyaları hazırlanacaktır

Timer modülünde 8 adet yazmaç bulunur. TIM_PRE yazmacı sayacın sistem saatinin kaç periyodunda değiştirileceğini belirler. Örneğin TIM_PRE yazmacına 1 yazılırsa sayaç sistem saatine göre 2 periyotta bir değişecektir. TIM_PRE yazmacına 0xFFFF_FFFF yazılırsa sayaç sistem saatinin 0x80000000 periyodunda bir değişecektir. TIM_ARE yazmacı sayacın ulaştığında sıfırlanacağı değeri tutar. Eğer sayaç aşağı sayıyorsa sayaç sıfıra ulaştığında TIM_ARE değeriyle tekrardan başlar. TIM_CLR yazmacının 0. biti TIM_CNT yazmacını kontrol eder: Eğer TIM_CLR[0] 1 ise TIM_CNT temizlenir. TIM_CLR yazmacının diğer bitleri etkisizdir. TIM_ENA yazmacının 0. biti sayacın sayma durumunu denetler. Eğer TIM_ENA[0] 1 ise sayaç saymaya devam eder; 0 ise sayaç önceki değerini korur, kalan bitler etkisizdir. TIM_MOD yazmacının 0. biti sayacın yönünü kontrol eder. Eğer TIM_MOD[0] 1 ise yukarı doğru, 0 ise aşağı doğru sayılacaktır. Diğer bitler etkisizdir. TIM_CNT yazmacında sayacın değeri tutulur. TIM_EVN yazmacı sayacın TIM_ARE değerine kaç kez ulaştığını tutar. Eğer aşağı yönde sayma işlemi yapılıyorsa sayacın kaç kez sıfıra ulaştığı bu yazmaca yazılır. TIM_EVC yazmacının 0. biti TIM_EVN yazmacını kontrol eder. TIM_EVC[0] 1 ise TIM_EVN yazmacındaki değer sıfırlanır.

Tasarlanmış olan Timer modülü bir AXI4-Lite slave arayüzüne sahiptir.

UART

UART çevre birimi, mikrodenetleyicinin üzerinde UART arayüzü bulunan cihazlarla haberleşmesini sağlayan birimdir. UART, eşzamansız (*-ing.* asynchronous) olarak çalıştığından bağlantılı bir saat hattı kullanılmaz. UART arayüzünde TX hattı verinin seri 1 baytlık paketler olarak gönderildiği, RX hattı ise gelen seri verinin çözüldüğü (*-ing.* deserialize) hattır. Bu iki hat birbirinden tamamen bağımsızdır, dolayısıyla eş zamanlı veri alışverişi gerçekleştirilebilir. Tasarlanmış olan UART modülünde veri boyutu 8 bit olup stop-bit sayısı 1, 1.5 veya 2 olarak ayarlanabilmektedir. Stop biti sayısının artırılması, veri alışverişi yapan cihazlara senkronizasyon için fazladan zaman tanıdığından daha güvenli veri iletimine imkan tanır. UART modülü, haberleşme hattında kullanılan baud rate'in en az iki katı frekansta çalışan bir saat sinyali ile zamanlamayı takip eder. Bu sayede UART modülü, gelen veriyi bit süresinin ortasına yakın, en kararlı noktada örnekleyerek zamanlama hatalarına karşı tolerans sağlar. Şartnamede belirtilmiş olan 1.5 stop-bit uygulaması da bu saat takibi stratejisi sayesinde gerçekleştirilir. UART çevre birimi için

raporun test kısmında da bahsedeceğimiz üzere birçok test gerçekleştirilmiştir. Testlerin yanı sıra birim, işlemci ile entegre edilmiş ve gerekli C ve Header dosyaları yazılmıştır [11].

UART modülü, içerisinde 5 adet kontrol yazmacı bulundurur:
UART_CPB, haberleşme sırasında kaç saat vuruşunda bir örnekleme yapılacağını ifade eder. Standart baud ratelerini elde etmek için gereken değerler, oluşturulan sürücü tarafından hesaplanarak haberleşme başlatılmadan önce UART_CPB yazmacına yazılır. UART_STP yazmacının [1:0] bitleri "00" ise stop-bit sayısı 1, "01" ise 1.5, "10" veya "11" ise 2 olur; geri kalan bitler anlamsızdır. UART_RDR yazmacı, alıcı modülü tarafından alınan verinin yazıldığı yazmaçtır. Alınan veri bu yazmacın [7:0] bitlerine yazılır, yazmacın diğer bitleri anlamsızdır. Veri alımı tamamlandığında UART_CFG[1] biti 1 yapılır. UART_TDR yazmacı modül tarafından gönderilecek veriyi [7:0] bitlerinde tutar, geri kalan bitler anlamsızdır. UART_CFG yazmacı, kontrol 8 bitlerini ve bayrakları (*-ing.* flag) bulundurur. UART_CFG[0] biti 1 ise UART_TDR yazmacında bulunan veri TX hattı üzerinden gönderilir. RX hattı üzerinden veri okuma işlemi tamamlandığında UART_CFG[1] biti 1 yapılır, TX hattı üzerinden veri gönderme işlemi tamamlandığında ise UART_CFG[2] biti 1 yapılır.

RX modülü, veri alımından sorumlu olduğu için çalışması sırasında yapılandırma yazmaçlarındaki değişimlerden olumsuz bir biçimde etkilenmez. TX modülünün veri gönderim sürecinde iletti̇ği verinin değişmemesi için verinin tutulduğu yazmacın değeri sabit kalmalıdır. Benzer şekilde veri gönderim sürecinin yarıda kesilmesi de gönderimin yapılan cihazda beklenmeyen sonuçlara yol açabilir. Bu durumları önlemek için veri aktarımı sırasında TX alt modülünün durumu kontrol edilir. Bu kontrolü gerçekleştirmek için AXI4-Lite arayüzü TX alt modülünden alınan tx_free sinyalini kullanır. Bu sinyalin 0 olması, TX alt modülü tarafından o sırada bir gönderim yapıldığını ifade eder. İşlemciden gelen okuma istekleri bu sinyalden bağımsız olarak her zaman yanıtlanmaktadır. Okunan değerlerin geçerli olup olmadığı bilgisi ise UART sürücüsü tarafından UART_CFG yazmacındaki bayraklar kontrol edilerek öğrenilir. İşlemciden gelen yazma istekleri ise eğer tx_free sinyali 1 ise yanıtlanır.

Tasarlanmış olan UART modülü AXI4-Lite slave arayüzüne sahiptir.

I2C Master

I2C (Inter-Integrated Circuit) Master modülü, mikrodenetleyicinin I2C slave modülü bulunduran cihazlarla AXI4-Lite arayüzü aracılığıyla iletişim kurmasını sağlar. I2C, SDA (Serial Data Line) ve SCL (Serial Clock Line) olmak üzere iki sinyal kullanarak seri (*-ing.* serial) veri aktarımı yapar. SDA sinyali adres ve veri aktarmak için kullanılırken SCL sinyali ise SDA sinyalinin zamanlaması için kullanılır.

Son olarak I2C_CFG (Configuration) I2C Master modülünde 5 adet yazmaç bulunmaktadır:

1) *I2C_NBY:* Slave cihazına yazılacak olan ya da slave cihazdan okunacak bayt sayısını tutar. Veri yolu maksimum 32 bit aktarımı desteklediğinden bu değer 1 ve 4 bayt aralığında olmalıdır. Eğer bu yazmaca yazılacak değer bu aralığın dışında ise bu aralıktaki en yakın sayıya yuvarlanacaktır: 0 durumunda 1'e, 4'ten büyük bir değer durumunda ise 4'e.

2) *I2C_ADR:* Slave cihazı seçimi için gerekli olan adres değerini tutar. 7-bit mod desteklendiğinden 0 ile 6. bitleri arası anlamlıdır.

3) *I2C_RDR:* I2C Master modülümünün slave cihazından okuyacağı verileri tutar. Bu verilerin boyutu I2C_NBY'nin değerine göre 1 ile 4 bayt arasındadır. Gelen veriler en anlamsız bit ilk (*-ing.* least significant bit first) olacak şekilde okunur.

4) *I2C_TDR:* I2C Master modülümünün slave cihazına yazacağı veriyi bulundurur. I2C_RDR'ye benzer olarak yazacağı verinin boyutu 1 ile 4 bayt arasındadır ve keza yine en anlamsız bit ilk olacak şekilde slave cihaza yazılır.

5) *I2C_CFG :* Bu yazmacının bitleri ise sırasıyla verilen amaçlarla kullanılmıştır: I2C_CFG[0] biti 1 olduğu süre boyunca slave cihaza veri aktarımı gerçekleşir. I2C_CFG[1] veri aktarımı tamamlandıktan sonra 1'e çekilir. I2C_CFG[2] biti 1 olduğu sürece slave cihazından veri okunur. I2C_CFG[3] ise okuma işlemi tamamlandıktan sonra 1 değerini alır.

Ayrıca şartname isterlerine ek olarak ekibimizce alınan karar üzerine I2C_CFG[4] biti ERROR durumları için kullanılmıştır. Gerçekleştirilen işlem sırasında slave cihazdan tanıma (*-ing.* acknowledge, ACK) sinyali gelmemesi durumunda 1'e çekilir. Son olarak I2C Master modülü aynı anda hem okuma hem de yazma işlemi yapmaz, buna donanımsal olarak önlem olmak için I2C_CFG[0] ve I2C_CFG[2] bitleri aynı anda 1'e çekilirse I2C_CFG[0] biti geçerli olur ve I2C_CFG[2] biti 0'a çekilir.

QSPI Master

QSPI (Quad Serial Peripheral Interface) Master modülü mikrodenetleyicinin QSPI arayüzünü destekleyen flash belleklerle veri alışverişi sağlayan bir modüldür. QSPI Master modülü SPI modülleri gibi CS ve SCLK hatlarına sahiptir. Ayrıca SPI modüllerinden farklı olarak IO0, IO1, IO2, IO3 hatlarına sahiptir. Dörtlü modda (*-ing.* quad) tüm bu hatlardan veri aktarımı gerçekleştirerek SPI modülünden daha hızlı veri alışverişi sağlar. QSPI Master modülünün iletişim kuracağı flash bellek MT25QL256ABA8E12 [12] olarak seçilmiştir. Bu flash yarışma şartnamesinde örnek olarak belirtilmesinden ve Verilog simülasyonunun yarışma şartnamesinde verilmesinden dolayı seçilmiştir. x1 veri genişliği Extended SPI, x2 veri genişliği Dual SPI, x4 veri genişliği ise Quad SPI protokolüne uygun olacak şekilde tasarlanmıştır. x1 veri genişliğinde buyruk ve adres gönderimi ile veri alışverişi IO0 hattı MOSI, IO1 hattı MISO olacak şekilde alışveriş işlemleri yapılır. QOR DOR ve QPP komutlarında hızlı veri aktarımının sağlanabilmesi için veri alışverişi sadece tek hat üzerinden değil DOR komutunda 2, QOR ve QPP komutlarında 4 hattan gerçekleşir. x2 veri genişliğinde buyruk, adres ve veri alışverişi IO0 ve IO1 hatları üzerinden gerçekleştirilir. QSPI Master modülünün x2 modunda flash ile doğru iletişim kurabilmesi için flash belleğin Dual SPI moduna alınması gerekmektedir. Flash belleği Dual SPI moduna almak için flash belleğin yapılandırma yazmacının (*-ing.* configuration register) 2. bitine 1 yazılması gerekmektedir bu işlem WRCR komutu ile yapılır. Aynı işlem QSPI Master modülü x4 veri genişliğinde yani Quad SPI modunda çalıştırılmak istendiğinde de yapılmalıdır. x2 modundan farklı olarak yapılandırma yazmacının 3. bitine 1 yazılmalıdır. x4 veri genişliğinde buyruk, adres ve veri IO0, IO1, IO2, IO3 hatlarından gönderilecektir. QSPI Master modülü için gerekli C ve header dosyaları yazılmıştır. QSPI Master modülünün sisteme entegrasyonu tamamlanmıştır.

QSPI Master modülü içinde 5 adet yazmaç bulunmaktadır. QSPI_CCR yazmacı QSPI Master modülünün yapılandırılmasında kullanılır. QSPI_CCR[7:0] flasha gönderilecek komut değerini tutar. QSPI_CCR[9:8] veri modunu tutar. 01 ise x1, 10 ise x2, 11 ise x4 modunda modül çalışır. QSPI_CCR[10] 0 ise flashtan okuma, 1 ise flasha yazma işlemi yapılır. QSPI_CCR[15:11] modülün kaç boş çevrim (*-ing.* dummy cycle) bekleyeceğini tutar. Boş çevrim çok hızlı okuma yapan komutlarda flashın veriyi hazırlayabilmesi için modülün hiçbir işlem yapmadan beklemesini sağlar. QSPI_CCR[23:16] bitlerindeki değerin 1 fazlası kadar bayt veri alışverişi yapılır. QSPI_CCR[24] flasha gönderilecek adresin 32 veya 24 bit olduğunu tutar. Bu yazmaç şartnamede rezerve olarak belirtilmiştir ve takımımız tarafından 3-4 bayt adresleme arası geçişte modüle bilgi vermek için kullanılmıştır. QSPI_CCR[24] 0 ise 24 bit, 1 ise 32 bit adresleme kullanılacaktır. QSPI_CCR[30:25] bitlerindeki değerin 1 fazlası kadar saat bölünerek SCLK sinyali ve modülün kendi içinde kullanacağı saat tickleri üretilir. QSPI_CCR[31] biti 1 ise QSPI_STA yazmacındaki değer sıfırlanır. QSPI_ADR yazmacında flasha gönderilecek adres bulunur. Eğer 24 bit adresleme modu açıksa QSPI_ADR[23:0] bitleri flasha gönderilecektir. QSPI_DR yazmacı RX ve TX olmak üzere iki adet fifoyla beraber çalışır. QSPI_DR yazmacından okuma yapmak istenildiğinde RX fifodaki değer okunur, bu yazmaca bir değer yazıldığında ise bu değer TX fifoya yazılır. QSPI_STA yazmacında modülün durumu ile ilgili bilgiler bulunur. QSPI_STA[0] biti 1 ise modül işlemleri tamamlamıştır 0 ise moül hala çalışıyordur. QSPI_STA[1] biti 1 ise modül meşguldür 0 ise modül boştadır. QSPI_STA[4] biti 1 ise RX fifo doludur. QSPI_STA[5] biti 1 ise RX fifo boştur. QSPI_STA[6] biti 1 ise TX fifo doludur. QSPI_STA[7] biti 1 ise TX fifo boştur. QSPI_STA[11:8] bitlerinde fifo hata durumları yer almaktadır. QSPI_STA[8] 1 ise RX fifo boşken okunmaya çalışıldı anlamına QSPI_STA[9] TX fifo doluyken yazılmaya çalışıldı anlamına gelir. Şartnamede istenen bu hata durumları dışında takımımız TX fifo boşken modül içinde okunmaya çalışılmasını yani TX underflow durumunu ve RX fifo doluyken modül içinde RX fifoya yazmaya çalışma durumunu yani RX overflow durumunu hata olarak tanımlamıştır. QSPI_STA[10] 1 TX underflow hatası vardır. QSPI_STA[11] 1 ise RX overflow hatası vardır. QSPI_STA yazmacının diğer bitleri etkisizdir. QSPI_FCR yazmacı RX ve TX fifolarını kontrol eder. QSPI_FCR[0] 1 ise RX fifo boşaltılır ve QSPI_FCR[0] 0'a çekilir. QSPI_FCR[1] 1 ise TX fifo boşaltılır ve QSPI_FCR[1] 0'a çekilir. QSPI_FCR yazmacının diğer bitleri etkisizdir.

Tasarlanmış olan QSPI Master modülü bir AXI4-Lite slave arayüzüne sahiptir.

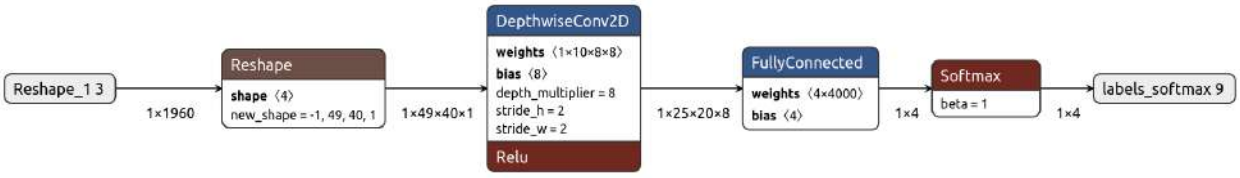
2.2.3. YZ Hızlandırıcı Tasarım Detayları

Bu bölümde, yarışma kapsamında geliştirilen YZ Hızlandırıcı tasarımı detaylandırılmaktadır. Bu kapsamda bölümün içeriği şu başlıklardan oluşmaktadır: 1-TFLite Micro Speech Modeli: Bu başlık altında RTL tasarımı gerçekleştirilecek olan yazılım modelinin amacı, katmanlarının yapısı ve özellikleri anlatılacaktır. 2-RTL Tasarım Akışı: Bu başlık altında planlanan YZ Hızlandırıcı mimarisi anlatılacak, yazılan modüller ve işleyişleri hakkında bilgi verilecek, karşılaşılan sorunlara karşı üretilen çözümler anlatılacaktır. 3-YZ Hızlandırıcı Veri Akışı: Bu başlık altında YZ Hızlandırıcı kullanılarak yapılacak çıkarım işleminde verinin izleyeceği akış anlatılacaktır.

Hızlandırıcı ile ilgili dosyalara Github sayfasından ulaşılabilir [13].

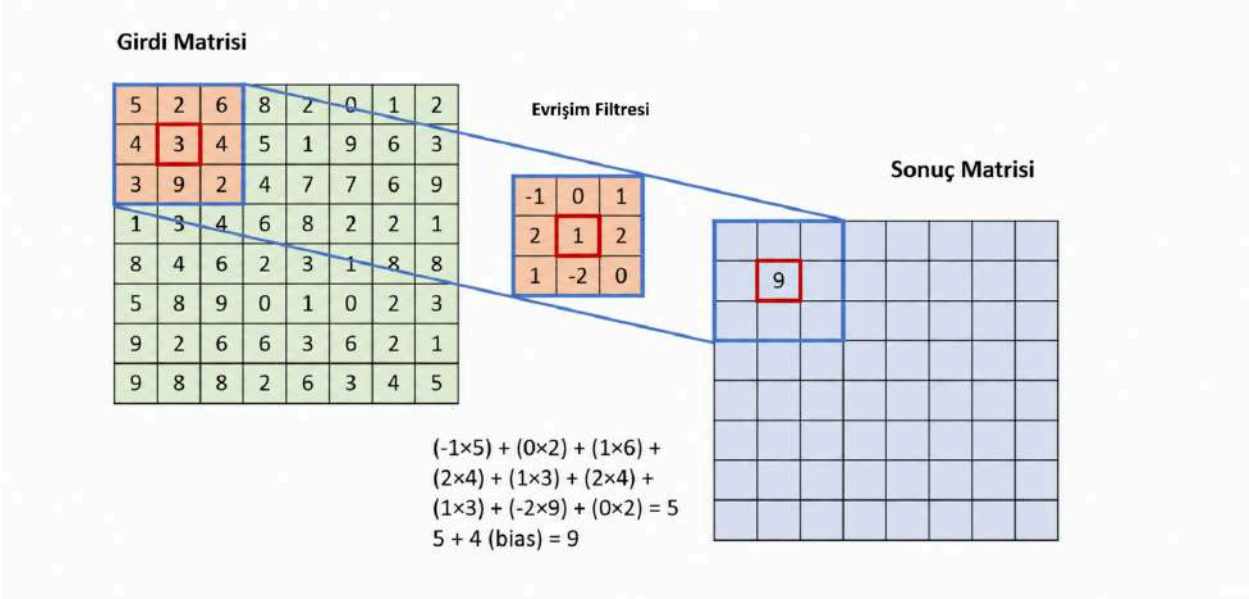
2.2.3.1 TFLite Micro Speech Modeli

Yarışma şartnamesinde bulunan YZ Hızlandırıcı isterlerine uygun şekilde TFLite Micro Speech modelini RTL düzeyinde gerçekleyecek bir hızlandırıcı birimi tasarlanması amaçlanmaktadır [14]. Model "Tiny Conv" mimarisine sahip olup bir adet 2-B evrişim (*-ing*. convolution) katmanı, bir adet tam bağımlı (*-ing*. fully connected) katmandan oluşmaktadır. Modelin ön işlemden (*-ing*. preprocessing) geçmiş ses verisi; "yes", "no", "bilinmeyen kelime" veya "sessizlik" olarak sınıflandırılmaktadır.



Şekil 2.2.3.1: TFLite Micro Speech Modeli [15].

Modelin, Netron aracı ile görselleştirilmiş hali Şekil 2.2.3.1 de verilmiştir. Model giriş olarak ses ön işleyicisinin (*-ing.* audio preprocessor) ürettiği 49x40 olarak yeniden şekillendirilmiş (*-ing.* reshaped) veriyi alır ve 2 boyutlu evrişim katmanına girdi (*-ing.* input) olarak verir. Evrişim işlemi evrişimli sinir ağlarında (*-ing.* convolutional neural networks) kullanılan verinin iki boyutlu yapısını bozmadan öznitelik çıkarmak (*-ing*. feature extraction) için kullanılan bir yöntemdir. Bu işlem sırasında şu adımlar gerçekleştirilir: 1-kernel matrisi, source layer matrisi üzerinde belirli adımlar (*-ing.* stride) boyunca yerleştirilir. 2-Kernel değerlerinin karşısına gelen sayılarla çarpılır. 3- Çarpım sonuçları birbirleriyle toplanır. 4- Son adımda toplanan değerlere sapma (*-ing.* bias) değerleri de eklenerek sonuç matrisinin değerleri hesaplanır. Şekil 2.2.3.2'de işlem detaylı olarak görselleştirilmiştir.



Şekil 2.2.3.2: Evrişim İşlemi Görseli ([16]'ten uyarlanmıştır).

Micro Speech modelinde, model eğitimi sırasında ağırlıkları belirlenmiş olan 10x8 boyutlarında olan 8 farklı filtre/çekirdek (*-ing.* kernel) kullanılarak 2 birimlik adımlar ile evrişim işlemi yapılmaktadır. Evrişim işleminin sonucu olarak 25x20 boyutunda 8 adet matris elde edilmektedir, modele doğrusalsızlık (*-ing*. non-linearity) katmak için denklem 1'de görülen ReLU (Rectified Linear Unit) aktivasyon fonksiyonu bu matrislerdeki verilere uygulanır.

$$F(x) = max(0, x) \quad (1)$$

ReLU işleminden sonra elde edilen veri düzleştirilip (*-ing.* flatten) tam bağımlı katmana girdi olarak verilir. Tam bağımlı katman girdi vektörü ile 4000x4 boyutundaki ağırlık matrisi arasında matris çarpımı gerçekleştirir. Çarpım sonucu elde edilen 4 sonuç değeri softmax katmanından geçirilerek "yes", "no", "bilinmeyen kelime" ve "sessizlik" sınıflarını (*ing.* class) temsil eden olasılık değerlerine dönüştürülür.

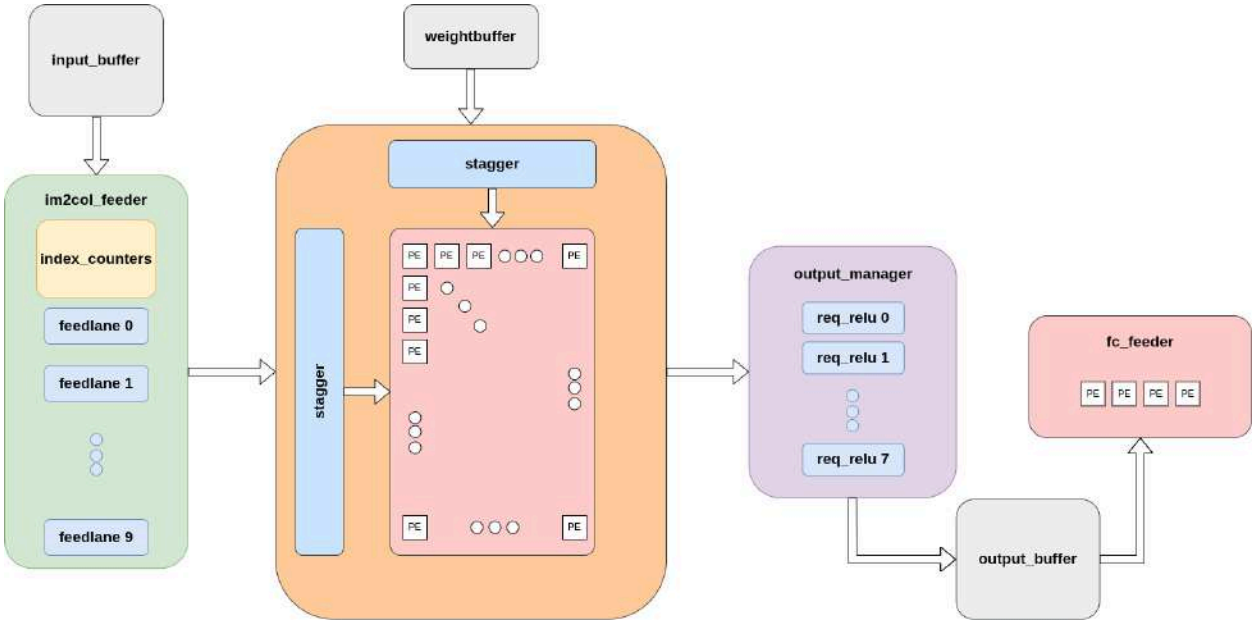
Yazılım modelini RTL üzerinde gerçeklemek için model üzerinden ağırlık değerlerinin, sapma değerlerinin ve Yeniden Kuantizasyon (*-ing.* requantization) değerlerinin elde edilmesi gerekmektedir. Bu değerler Micro Speech modelinin GitHub projesindeki .tflite dosyasından ai_edge_litert kütüphanesi kullanılarak yazılan bir Python programı ile elde edilmiştir. Ayrıca Netron [17] ve google-ai-edge model-explorer [18] kullanılarak model incelenmiştir, model akışı ve ağırlık değerleri kontrol edilmiştir.

2.2.3.2 RTL Tasarım Akışı

Bu kısımda Micro Speech modelinin RTL tasarım akışı, tasarlanan modüller ve veri akışları anlatılmaktadır.

Hızlandırıcı biriminin RTL tasarımları henüz son halini almamıştır. Tasarımda işlem birimi (*-ing.* processing element) ve sistolik dizi gibi temel işlem birimlerinden başlanılarak daha üst modüllere doğru giden içten dışa doğru bir tasarım uygulanmıştır. Şartname isterlerine göre RTL düzeyinde gerçeklenen model RISC-V çekirdeği (*-ing.* core) üzerinde çalışan yazılım gerçeklemesine kıyasla hızlanma elde etmeli ve yazılım gerçeklemesindeki modelin doğruluğunu %10'luk bir pencere dahilinde yakalamalıdır.

Model mimarisi incelendiğinde, donanımsal hızlandırmadan en fazla fayda sağlayabilecek bölümlerin çok sayıda çarpma-toplama işlemi içeren evrişim ve tam bağımlı katmanlar olduğu görülmüştür. Evrişim aşamasında yaklaşık 320 bin, tam bağımlı katmanda ise yaklaşık 16 bin MAC (multiply-accumulate) işlemi gerçekleştirilmektedir. Bu nedenle hızlandırıcı tasarımı, bu işlemlerin sistolik dizi yapısı üzerinde paralel olarak yürütülmesine odaklanmaktadır. Böylece yazılım tabanlı çalışmada her işlem için oluşan komut getirme ve kontrol yükü azaltılarak çok sayıda MAC işleminin eş zamanlı gerçekleştirilmesi hedeflenmiştir. Gerçeklenen tasarımın şeması Şekil 2.2.3.3'de verilmiştir.

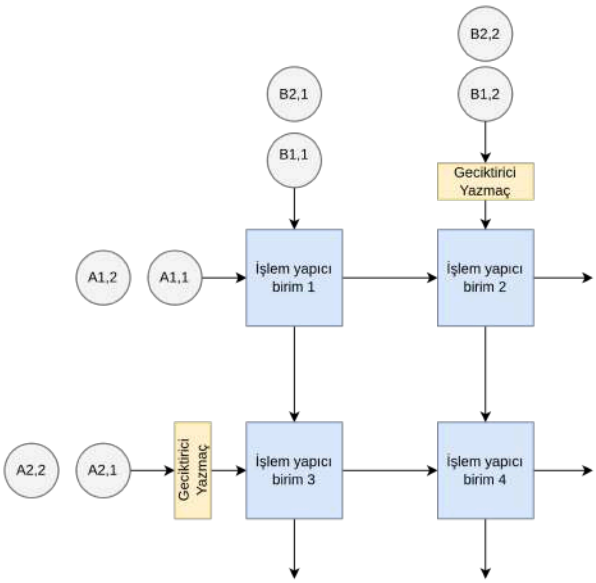


Şekil 2.2.3.3: Yapay Zekan Birimi RTL Tasarımı Akış Diyagramı

YZ Hızlandırıcı tasarımının gerçekleştiği aşamalar şu şekildedir:

1) Sistolik Dizi Tasarımı

Modelin hesaplama açısından yoğun bir kısmı olan evrişim işleminin yapılması için sektörde yaygın şekilde kullanılan ve çok sayıda arka arkaya yapılan MAC işlemi için verimli bir yapı sunan sistolik dizi (*-ing*. systolic array) yönteminin kullanılmasına karar verilmiştir.



Sistolik dizi, çarpma ve biriktirme yapan işlem birimlerinin bir dizi şeklinde birbirine bağlanması ve uygun şekilde beslenmesi sağlanarak oluşturulan bir GEMM (Generalized Matrix Multiplication) motoru işlevi görmektedir. Bir işlem birimi kendisine üstten ve soldan gelen verileri çarpıp içerisindeki biriktirici ile toplar ve sonraki çevrim üstten gelen veriyi alta, soldan gelen veriyi de sağa iletir. Sistolik dizinin satır ve sütunlarının başlarında verilerin doğru anda işlem birimlere girmelerini sağlayan geciktirici yazmaçlar bulunur. Bu yapı hızlı ve verimli matris çarpımı yapmayı amaçlamaktadır. 2x2 boyutunda örnek bir sistolik dizi görseli Şekil 2.2.3.4'te verilmiştir.

Şekil 2.2.3.4: 2x2 Sistolik Dizi Şeması

Micro Speech modelinde evrişim katmanı toplam MAC işlemi sayısının çok büyük bir kısmını oluşturduğu için sistolik dizi tasarımı evrişim işlemini hızlandırmaya odaklı olarak planlanmıştır. Sistolik diziler yaygın olarak iki işleyişe sahiptir:

1) İşlem birimlerinde çıktının sabit kalıp ağırlık değerlerinin her çevrim ilerlediği çıktısı sabit (*-ing.* output stationary) yapı. 2) Ağırlık değerlerinin sabit kalıp çıktı değerlerinin her çevrim ilerlediği ağırlığı sabittir. (*-ing.* weight stationary) yapı. Filtre boyutlarının küçük olması sebebiyle beslenmesinin zor olmayacağı öngörülmüş ayrıca 32 bit boyutundaki kısmi toplamların işlem birimleri arasında iletilmesi yerine 8 bitlik ağırlıkların iletilmesi için çıktısı sabit bir sistolik dizi modeline karar verilmiştir. Model evrişim katmanında 10x8 boyutunda 8 farklı filtre kullanmaktadır, matris çarpımı için sistolik dizinin boyutları, sütun sayısının filtre sayısına eşit olması için 10x8 olarak belirlenmiştir. Bunun sonucu olarak sistolik dizinin 8 sütununun her birine 8 filtreden biri verilecektir (*örn.* ilk sütundan ilk kernel, ikinci sütundan ikinci kernel verilir). 10 satırın her biri bir evrişim penceresini temsil etmektedir, satır sayısının 10 olması aynı anda 10 evrişim penceresinin işlenmesini sağlamaktadır. 25x20 boyutundaki çıktıyı oluştururken 10 pencere işlenerek iki seferde tam bir satırın bitirilmesini sağlayarak arabellekten okunacak adres üretimini kolaylaştırmaktadır. İşlem birimleri gelen verilerin çarpımını biriktirici yazmacına kaydetmektedir. Bitti sinyali geldiğinde ise işlem birimleri biriktirme yazmaçlarındaki değeri *RESERVE* yazmacına kopyalayıp output olarak sunmaktadır. Giriş verilerinden biri 0 olduğu zaman çarpma işleminin yapılmaması sayesinde güç tüketimini azaltmak için sıfır engellemesi (*-ing.* zero gating) yöntemi uygulanmaktadır. Sapma değerlerinin sonuçlara eklenmesi için biriktirme yazmaçlarının başlangıç değeri olarak belirlenmesi planlanmaktadır. Ayrıca tasarım satır sayısının artırılması ve adres üreticinin güncellenmesiyle geliştirmeye açık durumdadır.

Sistolik diziyi meydana getirecek *processing_element.sv, sa_top.sv, sa_wrapper.sv, stagge_sa_edge.sv* modülleri yazılmıştır ve vivado üzerinde testbench testleri aracılığıyla dalga formları da kontrol edilerek test edilmiştir. Modüllerin detayları şu şekildedir:

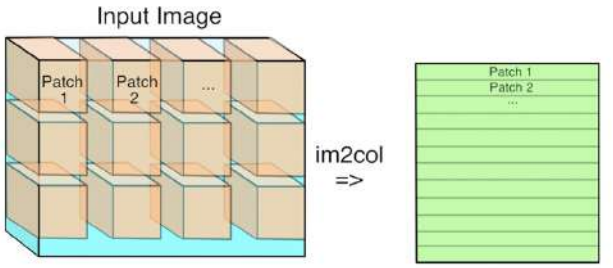
1. **processing_element:** Sistolik dizinin temel yapı birimidir, kendisine gelene iki veriyi çarpam işleminden geçirip biriktirici yazmacında tuttuğu değere ekler ve sonraki çevrim uygun çıkışlara gönderip yeni veriye aynı işlemi uygular. Bitti sinyali geldiği zaman biriktirici yazmacındaki değeri *RESERVE* yazmacına aktarır, çıkış portu üzerinden bu yazmaçtaki değeri dışarı verir.
2. **sa_top:** *processing_element* modüllerini gerçekler, aralarındaki kabloları oluşturur ve dizi yapısını kurar.
3. **stagge_sa_edge**: Sistolik diziye girecek verilerin basamaklı olarak geciktirilmiş şekilde girmesini sağlayan modüldür. Kendine sistolik diziye girecek satır ve sütunlar gelir, modül içerisindeki yazmaçlardan veriyi geçirerek *sa_top* modülüne veriyi uygun hale getirilmiş şekilde verir.
4. **sa_wrapper:** *sa_top* ve *stagger* modüllerini gerçekleyen üst modüldür. Oluşturduğu modüller arasındaki kabloları düzenler. Sistolik diziye girmek üzere kendisine gelen verileri önce stagger modüllerine verir, bu modüllerden çıkan veriyi de *sa_top* modülüne yönlendirir. processing_element modüllerinin output değerlerini dışarı yönlendirir.

2) Besleyici Tasarımları ve Evrişim işleminin Yapılması

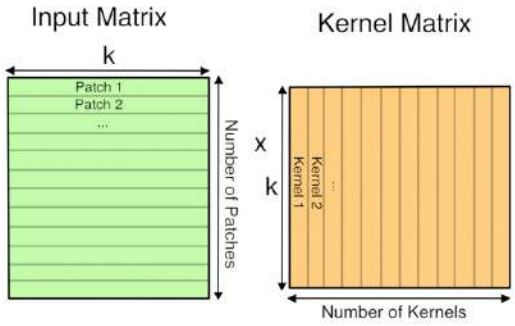
Sistolik diziyi oluşturan modüllerin yazılmasının ardından veriyi sistolik diziye yönlendirerek evrişim işleminin yapılmasını sağlayacak modüllerin tasarımına geçilmiştir. Verilerin satır ve sütunlara devamlı

şekilde verilmesi gerekmektedir, besleme sırasında boşluk oluşması durumunda sistolik dizi duraklamaktadır dolayısıyla da verim kaybı gerçekleşmektedir. Aynı zamanda doğru verilerin verilmesi gerekmektedir bunun için besleyici (*-ing.* feeder) modüller yazılmalıdır.

Sistolik dizi bir GEMM motorudur yani matris çarpımı yapmaktadır, bu yapıyı kullanarak evrişim işlemi yapmak için veriye im2col yöntemi uygulanmalıdır. im2col görüntüden-sütuna (*-ing.* image to column) anlamına gelmektedir ve evrişim işlemini standart matris çarpımına dönüştürmek için giriş matrisini yeniden düzenleyen bir veri dönüştürme tekniğidir. Evrişim işleminin yapılacağı her pencere düzleştirilip vektör haline getirilir ve yan yana dizilerek GEMM işlemine uygun bir matris elde edilir Şekil 2.2.3.5. Aynı zamanda evrişim işleminde kullanılacak filtrelerde düzleştirilip yan yana dizilerek uygun forma getirilir ve matris çarpımı yapılır Şekil 2.2.3.6.



Şekil 2.2.3.5: Veri Matrisi için im2col İşlemi [19].



Şekil 2.2.3.6: im2col İşlemi Uygulanmış Girdi ve Filtre Matrislerinin Çarpımı [19].

Evrişim pencereleri adım büyüklüğünün düşük olması sebebiyle büyük oranda üst üste binmektedir. Bu üst üste gelen kısımlar im2col işlemi esnasında tekrar yazıldığı için matris boyutu artmaktadır. Veri boyutundaki artışın önüne geçilmesi için matrisin arabellekten (*-ing.* buffer) okunduğu esnada on-the-fly im2col işlemi ile sistolik diziye verilmesine karar verilmiştir. Bunun için giriş verisi arabellekte orijinal halinde tutulacaktır, veriyi okuyan modüller, verinin sistolik diziye im2col işlemindeki sıraya uygun sırada verilmesini sağlayacaktır. Sistem tasarlanırken adres üretiminin nasıl yapılacağı, sistolik dizinin bekletilmeden nasıl besleneceği gibi sorunlarla karşılaşılmıştır. Benzer bir yapı kuran bir makale incelendiğinde sistolik dizinin her bir satırı için gerekli verileri depolayan ve bir adet adres üretici birimden oluşacak bir tasarımla on-the-fly im2col işlemi yapılabildiği görülmüştür [20]. Bu tasarım örnek alınarak tasarlanan besleyici modüllerinin içeriği şu şekildedir:

1. **im2col_feeder:** *address_generator* modülünün ürettiği adresleri BRAM modülüne yönlendirir. Okunan verileri *feedlane* modüllerine iletir. *feed_lane* modüllerinden veri alıp sistolik diziye göndermeye hazır olduğu zaman ready sinyali verir, *feed_lane* modüllerinin valid sinyali vermesi durumunda el sıkışma (*-ing.* handshake) gerçekleştirerek verileri kendi registerlarına kopyalar ve durum makinesinin durumunu değiştirerek verileri sırayla sistolik diziye gönderir.

2. **address_generator:** Doğru verilerin okunması için gerekli adresleri üretip *im2col_feeder* modülüne verir. İçerisindeki yazmaçlardaki satır/sütun değerleri ile okunması gereken adresi hesaplar. Bir durum makinesi ile adres üretimini kontrol eder.
3. **feed_lane:** *im2col_feeder* modülü içerisinde her sistolik dizi satırı için bir tane olmak üzere 10 tane feed_lane modülü oluşturulmaktadır. BRAM modülünden okunan veri bu modüllere verilir, her feedlane kendi evrişim penceresine ait verileri seçmek için bir offset mekanizması kullanır. Bütün modülleri veri almaya hazır olduğu zaman *im2col_feeder* modülünden gelen sinyalle durum makinesi değişiklikleri yapılır ve veri okunmaya başlar. Her biri kendi 10x8 boyutundaki evrişim penceresinin bir satırındaki verileri kaydeder, hepsi hazır olduğunda *im2col_feeder* modülü ile valid-ready el sıkışması gerçekleştirilir ve sıradaki satırın verileri kaydedilmeye başlanır.

Filtreler ise veri matrisine göre daha küçük boyutlu olduğu için verilerini yazmaç dizisinde tutacak bir modül ile ağırlıklar sistolik dizinin sütunlarına beslenecektir.

1. **weight_feeder:** 8 filtrenin ağırlıklarını düzleştirilmiş halde yazmaçlarında tutar. *im2col_feeder* modülünden aldığı sinyalle sistolik diziye ağırlıkları gönderir.

10 pencerelik bir kısmın satırlarının tamamı bittiğinde *im2col_feeder* modülü son veriyle birlikte "bitti" sinyali gönderir, bu sinyal sistolik dizi boyunca ilerler ve geçtiği processing element modüllerindeki hesaplanan değerleri *RESERVE* yazmaçlarına aktarmalaını sağlar. Sinyal sistolik dizinin sağ alt işlem yapıcı biriminden çıktığı zaman üretilen sonuçları toplama sinyali olarak output manager modülüne iletilir.

3) ReLU, Requantization ve Tam Bağımlı Katman İşlemlerinin Yapılması

Evrişim işlemi sonucu oluşan değerlerin sistolik diziden toplanması gerekmektedir. Verilerin toplanması sırasında sistolik dizinin durdurulması ciddi bir verim kaybına yol açabilmektedir. Bu problemi çözmek için *processing_element* modüllerine eklenen *RESERVE* yazmaçlarından faydalanılmıştır. Çıktıları toplamak için işlem birimlerinin *RESERVE* yazmaçlarının bağlı olduğu bir *output_manager* modülü yazılmıştır. Toplanan değerlerin ReLU ve Yeniden Kuantizasyon işlemlerinden geçirilmesi için *relu_req* modülü yazılmıştır. ReLU ve Yeniden Kuantizasyon işlemlerini gerçekleştiren *relu_req* modülünün mevcut tasarımı kombinasyonel bir yapıya sahiptir, bu durumun kritik yol uzunluğunu artıracağı tahmin edilmektedir. Bu olası sorunun önüne geçilmesi için modülde gerçekleştirilen işlemler arasına yazmaçlar yerleştirilerek boru hattı kurulması planlanmaktadır. Yeniden Kuantize edilmiş veriler bir çıktı arabelleğine yazıldıktan sonra *fc_manager* modülü tarafından okunarak tam bağımlı katman işlemlerinden geçirilmeye başlanır. Bu modüllerin detaylı anlatımı aşağıdaki gibidir:

1. **output_manager:** Sistolik dizinin 10 evrişim penceresini hesaplamayı bitirdiğini belirten bitti sinyalini alınca işlem birimlerindeki değerleri kendi içindeki yazmaçlara kopyalar. İçerisinde 8 adet *relu_req* modülü gerçekler. Topladığı sistolik dizi çıktılarını sekizerli gruplar halinde bu modüllere yönlendirir, *relu_req* modülleri işlemlerinin tamamlandığını belirten çıktı sinyali verince değerleri alır ve çıktı arabelleğine yazar.
2. **relu_req:** Aldığı değerlerin işaret (*-ing.* sign) bitine bakar ve negatif ise çıktı olarak 0 verir pozitif ise değeri değiştirmeden devam ettirir. Devam eden değer tekrar 8 bitlik integer formatına

dönüştürülmesi için Yeniden Kuantizasyon (*-ing.* requantization) işleminden geçirilir. Bu işlem sonucu 0-255 değerleri arasına indirgenen sonuçlarla birlikte işlemin bittiğini gösteren valid sinyali de çıktı olarak verilir.

3. **fc_manager:** İçerisinde 4 işlem biriminden oluşan 1x4 boyutunda küçük bir sistolik dizi oluşturur. Tam bağımlı katmanda yapılması gereken matris çarpımında kullanılacak 4000x4 boyutundaki ağırlık değerlerinin her bir sütunu işlem birimlerinden birine beslenir. Evrişim işlemi sonrasında düzleştirilerek matris çarpımına girecek veri değerleri ise ilkinden başlayarak işlem birimleri boyunca ilerlerler. *output_manager* modülünün 10 pencerelik çıktıyı hazırlaması sonucu sinyal göndermesiyle hazır olan değerleri işlemden geçirir ve sıradaki değerleri bekler. Bu sayede bütün evrişim işleminin bitmesini beklemeden çalışmaya başlar.

Mevcut tasarımda *fc_manager* modülünün bütün evrişim işleminin bitmesini beklemesinin önüne geçilmesi için *output_manager* modülünden veriler çıktıkça mevcut verileri işleyip sıradakileri beklediği bir tasarım oluşturulmuştur. Bu sayede evrişim işlemi ile aynı anda tam bağımlı katman da çalışması sağlanıp verimin artırılması planlanmıştır. Çıktı arabelleğinin boyutu da bu sisteme uygun olacak şekilde belirlenecektir.

4) Verinin Arabelleğe Aktarılması ve Sonucun Üretilmesi

Giriş verisinin YZ Hızlandırıcı birimine ayrılmış bellek alanından hızlandırıcı üzerine çekilmesi ve giriş verisi arabelleğine kaydedilmesini içeren kısım şu şekilde planlanmaktadır:

1. Bellekte bulunan veri AXI arayüzü kullanılarak UART-stream modülü tarafından YZ Hızlandırıcı üzerine alınacaktır.
2. Veri padding işlemi yapılmış halde arabellek modülüne yazılacaktır.

Arabellekteki veri, im2col_feeder modülü ile okunmaya başlanıp evrişim ve tam bağımlı katman işlemlerini yapan birimlerden geçer. Devamında softmax işlemini uygulayacak ve çıkarım işleminin sonucunu üretecek modüllere yönlendirilir. Hızlandırıcı veri akışı tamamlandıktan ve sınıflandırma sonucu üretildiğinde CV32E40P çekirdeği, bir CPU kesme (*-ing.* interrupt) sinyali ile bilgilendirilecektir.

2.2.3.3 YZ Hızlandırıcı Veri Akışı

YZ Hızlandırıcı çıkarım sürecinde veri akışı şu aşamalardan geçmektedir:

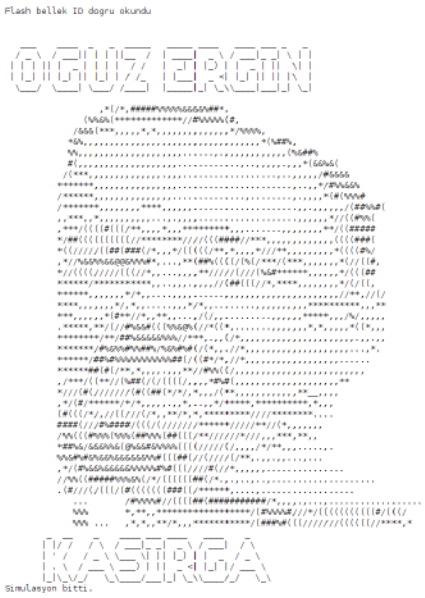
1. **Çıkarım Yapılacak Verinin YZ Hızlandırıcı Bellek Alanına Yazılması:** Veri akışında kullanılacak UART ve YZ Hızlandırıcı modüllerinin kontrol durum yazmaçlarının ayarlanmasıyla çıkarım süreci başlayacaktır. UART-Stream çevre birimi çıkarım yapılacak veriyi hızlandırıcı bellek adresine yazacaktır.
2. **Verinin YZ Hızlandırıcı Üzerine Alınması:** Çıkarım yapılacak veri hızlandırıcıya ait bellek alanından AXI arayüzü üzerinden hızlandırıcı üzerine alınacaktır. Veri, YZ Hızlandırıcı birimindeki girdi verisi ara belleğine, padding işlemi yapılmış şekilde yazılmış şekilde bulunur. Hızlandırıcı üzerine alınmış olan veri çıkarım işlemi için gerekli işlemlerden geçer.

3. **Sonucun Üretilmesi:** Çıkarım işlemi tamamlandıktan sonra sonuç belleğe yazılacaktır. Ardından çekirdeğe kesme sinyali gönderilecek ve çekirdeğin interrupt service routine'ini çalıştırmasıyla çıkarım sonucunun UART üzerinden ekrana basılıp veri akışı tamamlanmış olacaktır.

2.3. Boot

Ön yükleme (*-ing.* boot) işlemi için C dilinde bir önyükleyici (*-ing.* bootloader) kodu yazılıp riscv-gnu-toolchain aracıyla derlenmiştir. Bu derleme esnasında yığın işaretçisinin (*-ing.* stack pointer) değerinin bellek haritamıza uygun olarak doğru adrese (veri belleğinin sonuna 0x0000_5000) atanması için Assembly Kaynak Dosyası (*-ing.* Assembly Source File) yazılmıştır. Buyruk ve veri belleklerinin C kodu derlenirken derleyiciye (*-ing.* compiler) doğru tanıtılması içinde bağlayıcı betikler (*-ing.* linker script) kullanılmıştır yazılmıştır. Projede 2 adet bağlayıcı betik bulunmaktadır: *bootloader.ld* dosyası ön yükleyici kodu için kullanılırken *main.ld* dosyası flash belleğin içine yazıp çalıştırmak istediğimiz ana program için kullanılmaktadır.

Kaynak dosyaları, bağlayıcı betikler ve C kodları *script.sh* betiğiyle (*-ing.* script) derlenerek *bootloader_static.hex* dosyası ve flash belleğin içine yazılacak *flash.mem* dosyaları oluşturulur. *bootloader_static.hex* dosyası *boot.mem* dosyasının içine yazılır. *script.sh* dosyası Makefile dosyasını çalıştırır, gerekli kodları derler ve yardımcı Python betikleriyle çıkan ikili taban (*-ing.* binary) dosyalarını simülasyonda önyükleme belleğine (*-ing* boot rom) ve flash belleğin hafıza dosyalarına yüklenecek formata getirir. Python betiği ikili taban dosyalarını flash belleğe uygun hale getirirken flash belleğin 0x0000_1000 adresine kaç adet buyruk okunması gerektiğini yazar ve buyrukları 0x0000_1004 adresinden başlayarak flash bellek formatına dönüştürür. Aynı işlem veri belleğine yazılacak veriler için de tekrarlanır ve 0x0000_3000 adresinden başlayarak veri belleğine yazılacak kelime (*-ing.* word) sayısı ve veriler flash belleğin ilgili adresine yazılır.



Şekil 2.3.1: Örnek Program Çıktısı

Sistem ilk kez çalıştırıldığında QSPI Master modülü üzerinden öncelikle flash bellekten 0x0000_1000 adresinden 4 bayt okuma yapılır ve flash bellekten buyruk belleği için kaç okuma yapılacağı kaydedilir. Flash bellekten buyruklar READ komutu ile 4 baytlık bloklar halinde okunur ve bunlar buyruk belleğine yazılır. Buyruk belleğine yazma işlemi tamamlandıktan sonra flash bellekten 0x0000_3000 adresinden 4 bayt okuma yapılır ve flash bellekten veri belleği için kaç okuma yapılacağı kaydedilir. Flash bellekten veriler READ komutu ile 4 baytlık bloklar halinde okunur ve veri belleğine yazılır. Veri belleğine yazma işlemi tamamlandıktan sonra çekirdek buyruk belleğine atlatılır ve yürütme buyruk belleğine devredilir. Önyükleme işlemi bu şekilde tamamlanır. Önyükleme işleminin doğru çalıştığını test etmek için UART ve QSPI Master modüllerini kullanan bir program yazılmıştır. Önyükleme işlemi doğru bir şekilde gerçekleşmiştir. Çalıştırılan programın çıktısı Şekil 2.3.1'de bulunmaktadır. İlerleyen süreçte önyükleme işleminin hızlandırılabilmesi için READ komutu yerine QOR komutu kullanılarak 28 baytlık bloklar okunması hedeflenmektedir. Önyükleme ile ilgili bahsedilen dosyalar Github Repository'mizde bulunmaktadır [21].

2.4. FPGA Prototipleme

Projede tasarlanan mikrodenetleyici sisteminin doğrulanması amacıyla FPGA üzerinde bir prototipleme işlemi gerçekleştirilmiştir. Bu amaçla, donanım tasarımı Vivado kullanılarak SystemVerilog dilinde oluşturulmuştur. Tasarımın FPGA üzerinde çalışacak bit dosyası (*-ing.* bitstream), sentez ve implementasyon adımlarının tamamlanmasıyla üretilmiştir. Oluşturulan bit dosyası, USB bağlantısı ile Basys3 FPGA geliştirme kartına yüklenmiş ve sistem başarıyla programlanmıştır.

İlk olarak, kartın bağlı bulunduğu bilgisayarda çalıştırılan C kodu [22] ile karta UART üzerinden bir mesaj gönderilmiş ardından karta yüklediğimiz sistem sayesinde yine UART hattı üzerinden mesaj geri bilgisayara iletilmiş ve C kodu ile yakalanmıştır. Ekibimizce hazırlanan ve daha tutarlı bir test için uzun tutulan bir ASCII karakter dizisi UART üzerinden terminale çıktı olarak verilmiştir. Başarıyla elde edilen terminal çıktısı Şekil 2.4.1'de görülmektedir. Bu sonuç, tasarlanan mikrodenetleyicinin gerçek donanım üzerinde de düzgün çalıştığını göstermektedir. Aynı zamanda karmaşık bir ASCII deseninin sırasıyla basılması, işlemcinin program akışını doğru izlediğine ve bellekten komutları düzgün şekilde okuduğuna bir işarettir.

Daha sonra yapılan testlerde UART çevre birimine klavyeden girilen karakterler RX hattı üzerinden karta iletilmiş ve işlemci bu karakterleri doğru şekilde okuyabilmiştir. Bu sayede UART modülünün veri gönderme (TX) ve veri alma (RX) işlevi doğrulanmıştır. UART haberleşmesinin doğru ve sorunsuz bir şekilde gerçekleştiği testlerin sonucunda gözlemlenmiştir.



Şekil 2.4.1: UART FPGA Prototipleme Testi Terminal Çıktısı

3. Çip Tasarım Akışı

Mikrodenetleyicinin fiziksel tasarımı için sağlanacak olan ticari EDA (Electronic Design Automation) lisansının temini DTR aşamasında sonra gerçekleşeceği için bu aşamada açık kaynaklı bir yazılım olan OpenLane [8] kullanılmıştır. Bu yazılımın kullanılmasına Kasırga Laboratuvarı Slack grubunda gerçekleştirilen tartışmalar sonucunda karar verilmiştir. OpenLane; OpenROAD, Yosys, Magic, Netgen, CVC, SPEF-Extractor, KLayout ve bir dizi özel komut dosyası da dahil olmak üzere çeşitli bileşenlere dayanan, tasarım keşfi ve optimizasyonu için otomatikleştirilmiş bir RTL'den GDSII'ye akıştır. Bu akış, RTL'den GDSII'ye kadar tüm ASIC uygulama adımlarını gerçekleştirir ve aşağıda verilen şekilde ilerler:

Mantık Sentezi (*-ing.* logic synthesis): Bu aşamada RTL seviyesindeki kod yapısal olarak kontrol edilir ve eğer varsa hatalar tespit edilir. Herhangi bir hatanın olmadığı tespit edilirse RTL mantıksal hücrelere dönüştürülür. Ayrıca bir bağlantı listesi (*-ing*. netlist) de oluşturulur. Bu bağlantı listesi aracılığıyla ilk statik zamanlama analizi (*-ing.* static timing analysis) de gerçekleştirilir.

Yerleşim Planı (*-ing*. floor plan): Bir önceki adımda bağlantı listesi oluşturulduktan sonra bu adımda fiziksel planlama başlar. Çipin boyutu, kullanım oranı gibi temel özellikler belirlenir ve ardından giriş çıkış pinleri çipin üzerine yerleştirilir. Kısa devre oluşumunu önleyen tap hücreleri ve çipin ani akım ihtiyaçlarını karşılamak için decap kapasitörleri de eklendikten sonra son olarak güç dağıtımında sorumlu olan güç dağıtım ağı (*-ing.* power distribution network) çipin üzerine yerleştirilir.

Yerleştirme (*-ing*. placement): Bu aşama genel yerleştirme (*-ing.* global placement) ile başlar. Bu süreçte hücrelerin yaklaşık konumları belirlenir ve hücre yoğunluğu optimize edilmeye çalışılır. Ardından bir statik zamanlama analizi yapılır ve daha sonra yerleştirme boyutlandırıcısı (*-ing*. placement resizer)

adından bir optimizasyon uygulaması çalıştırılır. Yeniden boyutlandırıcı Nesterov'un hızlandırılmış gradyan inişine (*-ing.* accelerated gradient descent) [23] dayanan doğrusal olmayan (*-ing.* nonlinear) bir uygulamadır. Hücreleri örtüşmelerini (*-ing.* overlap) önleyecek şekilde dağıtır ve tel uzunluklarını (*-ing.* wire length) da en kısa hale getirir. Çipin son durumu kullanılarak bir statik zaman analizi daha gerçekleştirilir.

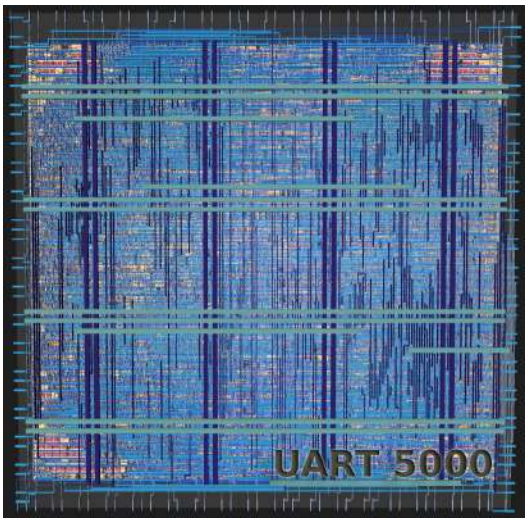
Saat Ağacı Sentezi (*-ing.* clock tree synthesis): Yerleştirmeden sonra saat ağı (*-ing.* clock network) tasarlanması ile akış devam eder. Saat sinyalinin devredeki elemanlara aynı anda ve yeterli güçte ulaşması amaçlanır. Ancak bir saat sinyalinin çok fazla devre elemanını sürmesi durumunda gerilim değerinde düşmeler meydana gelebilir ve bunun sonucu olarak devrede istenmeyen durumlar yaşanabilir. Bunu önlemek için ise tipik olarak dengeli bir ikili ağaç (*-ing.* balanced binary tree) kullanılır. Ağaç oluşturulurken dallanma noktalarına arabellekler eklenir ve saat hatları yönlendirilir ve bu sayede daha önce bahsedilmiş olan istenmeyen durumlar engellenebilir. Ardından statik zaman analizi yapılır ve tekrardan yerleştirme boyutlandırması yapılarak hücre boyutları yeniden optimize edilir. Ayrıca dengeli ikili ağaç yerine H ağaçları (*-ing.* H trees) gibi daha özgün ağaçlar da kullanılabilir.

Bağlantı (*-ing*. routing): Bu aşamada çipin üzerindeki hücreler birbirlerine fiziksel ve elektriksel olarak bağlanırlar. Hücreler arasındaki bağlantıların sıkışmaması için öncelikle bağlantı sonrası zamanlama optimizasyonları (*-ing.* post-routing timing optimization) ile sinyal gecikmeleri gözetilerek son ayarlamalar yapılır. Ardından statik zaman analizi yapılır. Daha sonra hücre boyutlarını sinyal gecikmeleri gözetilerek tekrar ayarlamak için zaman odaklı bağlantı boyutlandırması (*-ing.* router resizer timing) işlemi yapılır. Bundan sonra bir statik zaman analizi daha yapılır. Bu analizin ardından çipi elektrostatik deşarj ve anten etkisi gibi risklerden korumak amacıyla tasarıma diyot eklenmesi (*-ing.* diode insertion) işlemi gerçekleştirilir ve hücre koordinatları son kez sabitlenir. Buraya kadar yapılan hazırlıkların ardından fiziksel yolları yaklaşık olarak belirlemek için genel bağlantı (*-ing.* global routing) adımına geçilir. Anten özelliği gösteren yolların düzeltilmesinin (-ing. global routing repair antennas) ardından bu aşama için son bir kez daha statik zaman analizi gerçekleştirilir. Sonrasında çip yüzeyindeki metal yoğunluğunu fiziksel kurallara uygun hale getiren dolgu ekleme (-ing. fill insertion) işlemi de tamamlanır nihai aşama olan detaylı bağlantıya (*-ing.* detailed routing) geçilir. Detaylı bağlantı hücreler arasındaki yolları belirler. Bu sırada metal katmanlarına yollar çizilir ve gerekli olan yerlerde katmanlar arasında geçiş sağlanması için gerekli olan metal ara bağlantılar (-ing. via) da eklenir. A* [24] veya Lee [25] gibi labirent yönlendirme algoritmalarının kullanıldığı bu adım eğer ilk seferde yolların tamamı çizemezse kendini tekrar tekrar çağırarak yolları optimize eder.

Son Doğrulama (*-ing*. sign-off): Bu son aşamada çipin üretime tam anlamıyla hazır olup olmadığı denetlenir. İlk olarak tasarımın zamanlama kısıtlarına uyup uymadığı kesinleştirmek için son kez statik zaman analizi yapılır. Eğer bu analiz sonucunda herhangi bir zamanlama ihlali tespit edilirse Mühendislik Değişiklik Emri (*-ing*. Engineering Change Order, ECO) döngüleri devreye girer. Bu döngüler, kritik yollara (*-ing*. critical path) arabellek ekleyerek veya hücre boyutlarını yeniden ayarlayarak zamanlama hatalarını giderir ve tasarımı istenen çalışma frekansına uygun hale getirir. Zamanlama optimizasyonları tamamlandıktan sonra fiziksel doğrulama adımları başlar. Öncelikle Magic veya KLayout gibi programlar kullanılarak çipin katman katman 2 boyutlu geometrik çizimlerini barındıran GDSII dosyaları oluşturulur. Daha sonra, tasarım kural kontrolü (*-ing.* Design Rule Check, DRC) işlemi gerçekleştirilerek hat

uzunlukları ve hatlar arası minimum mesafe gibi belirlenmiş olan fiziksel kısıtlamalara uyulup uyulmadığı denetlenir. Ardından layout-şema karşılaştırmasına (*-ing.* Layout versus Schematic, LVS) geçilir; bu işlemde, GDSII dosyasından çıkarılan bağlantı listesi ile orijinal mantıksal tasarımdan gelen bağlantı listesi kıyaslanarak tasarımın doğruluğu teyit edilir. Son olarak, üretim sırasında hücrelere zarar verebilecek yük birikimlerini önlemek amacıyla anten kontrolü yapılır. Tüm bu zorlu testlerin sıfır hata (DRC/LVS-clean) ile tamamlanmasının ardından, çip yerleşimi GDSII formatında dışa aktarılarak fiziksel üretim sürecine tamamen hazır hale gelmiş olur.

Projemizde çip tasarım akışı ilk olarak daha önceden doğrulanmış olan UART için gerçekleştirilmiştir. Tasarım için oluşturulmuş olan yapılandırma (*-ing.* config) dosyasının içerisinde çip alanı, hedef yoğunluk (*-ing*. density) oranı ve saat periyodu gibi önemli tasarım kısıtları tanımlanmıştır. UART çip tasarım akışından geçirilirken alan yetersizliği veya daha yüksek bir yoğunluğa ihtiyaç olduğuyla ilgili parametrik hatalarla karşılaşılmıştır. Yapılandırma dosyasına yapılan düzenlemeler ile bu problemler çözülmüştür. Farklı yoğunluk oranlar ve çip alanı değerleri denenmiş ve takımımızca en iyi performans veren değerler bulunmaya çalışılmıştır.



Şekil 3.1.1: OpenLane Çıktısı (GDSII)

ÖTR'mizde belirtilen azami değer olduğu için 20 nano saniyelik saat periyoduna (50 MHz'lik frekans) sahip olan, Şekil 3.1.1'de verilmiş olan çipin son doğrulama aşamasında elde edilen sonuçlar incelendiğinde tipik uç koşulda 10.76 nanosaniye kurulum zaman payı (*-ing.* setup slack) ölçülmüştür. Bu değer tasarımın büyük bir zamanlama marjına sahip olduğunu ve zamanlama gereksinimlerini güvenli bir şekilde karşıladığını göstermektedir. Toplam negatif zaman payı (*-ing*. Total Negative Slack, TNS) tüm zamanlama ihlali yapan yolların negatif zaman payı değerlerinin toplamıdır. En kötü negatif zaman payı (*-ing*. Worst Negative Slack, WNS) ise en büyük zamanlama ihlali yapan yolun negatif zaman payıdır. Toplam negatif zaman payı ve en kötü negatif zaman payı değerlerinin sıfır olması sistemde herhangi bir zamanlama ihlalinin bulunmadığını kanıtlamaktadır. 0.35 nanosaniye hold slack değeri de güvenli seviyededir. Güç analizi sonuçlarına göre, tasarımın ortalama güç tüketimi 2.34 miliwatt olarak ölçülmüştür. Bu değer sistemin güç profilini göstermekte olup düşük güçlü uygulamalar için uygun görünmektedir. Toplam güç tüketiminin yaklaşık %34.2'si sıralı mantığa (*-ing.* sequential logic) ve %65.8'i

birleşik mantığa (*-ing.* combinational logic) aittir. Saat ağı analizi sonucunda 0.26 nano saniyelik saat kayması (*-ing.* clock skew) elde edilmiştir, bu da saat sinyalinin devredeki elemanlara eşit dağıtıldığını göstermektedir. Bu implementasyon, 180×180 mikro metrekare boyutlarındaki kalıp (*-ing.* die) alanında herhangi bir tasarım kural kontrol hatası almadan başarıyla gerçekleştirilmiştir.

4. Test

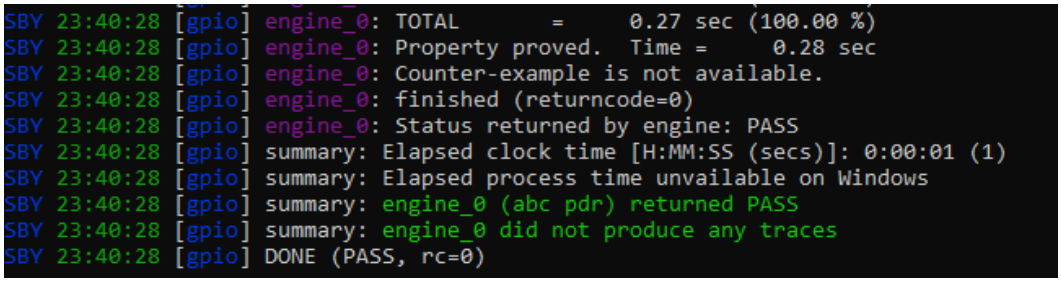
Projemizin tabi tutulduğu ve devam eden test sürecimiz modül düzeyinde başlayan ve entegre sistem seviyesine kadar uzanacak olan kapsamlı bir plandır. İlk olarak CV32E40P işlemci çekirdeğinin doğrulanması ve ardından çevre birimlerinin ilgili birim (*-ing.* unit) testleri tamamlanmıştır. Çevre birimlerinin sisteme entegre edilmesi ile oluşan sistemlerin testleri ise birimler sisteme entegre edildikçe devam etmektedir. DTR sonrası süreçte kod kapsamı (*-ing.* code coverage) ve fonksiyonel kapsam (*-ing.* functional coverage) testleri yapılması planlanmaktadır.

Modüller üzerinde yapılmış olan ve yapılacak olan testlerin tamamını kapsayan test planımız ise aşağıda açıklanmıştır:

İşlemci: İşlemcinin doğrulanması iki aşamada gerçekleştirilmiştir. İlk adımda CV32E40P çekirdeği Spike simülatörü kullanılarak doğrulanmıştır. Bu aşamayla ilgili dosyalar [4] dizininde yer almaktadır. Çekirdeğin doğrulanması için *test.c* [26] yazılmıştır ve bu kod riscv-gnu-toolchain kullanılarak derlenmiştir. Bu işlemin sonucunda oluşan buyruklar MemoryFiles klasöründe bulunan *INIT_FILE.mem* dosyasına yazılarak *cv32e40p_sim_ram.sv* modülü kullanılarak çekirdeğe bağlanmıştır. Ardından *tb_top.sv* testbench'i çalıştırılmış ve testbench içerisindeki izleyici (*-ing.* tracer) ile beraber çekirdeğin çalıştırdığı buyruklar *simulation_trace.log* dosyasına kaydedilmiştir. Aynı C kodu Spike kullanılarak çekirdekte çalıştırılmış ve sonuç *spike.log* dosyasına kaydedilmiştir. Bu iki log dosyasının *verify.py* Python betiğiyle aynı oldukları doğrulanmıştır. Akabinde bu betiğin sonucu *output.log* dosyasına yazılmıştır. İkinci aşamada ise CV32E40P çekirdeğine çevre birimleri entegre edilmiştir. QSPI Master modülü ile beraber önyükleme mekanizması kullanılarak işlemci flash bellekten buyrukları ve veriyi okumuş, flash bellekten okunan programda UART modülü kullanılarak veri gönderimi işlemini başarıyla tamamlamıştır. Bu iki testle beraber işlemcinin istenilen gibi çalıştığı doğrulanmıştır. DTR sonrasındaki süreçte Coremark testi yapılarak işlemcinin doğrulanması için yapılan test sayısının arttırılması planlanmaktadır.

AXI4-Lite Slave Arayüzü: AXI4-Lite slave arayüzünün doğrulaması biçimsel doğrulama (*-ing.* formal verification) ile kanıtlanmıştır.

AXI4-Lite arayüzünün işleyişine dair doğrulama ifadeleri (*-ing.* assertion) bulunduran bir modül [27] ile AXI4-Lite slave modülü bağlanıp daha sonra symbiyosys [28] aracı kullanılarak test edilmiştir. Şekil 4.1.1'de de görüldüğü üzere, arayüzün hiçbir saat döngüsünde protokol kurallarını ihlal etmediği ve tüm doğrulama ifadelerini doğru bir şekilde sağladığı matematiksel olarak ispatlanmıştır.



Şekil 4.1.1: Biçimsel Doğrulama Log Çıktısı

Yarışma takvimi doğrultusunda, ticari EDA lisanslarının sağlanmasının ardından final tasarımı sürecinde AXI4-Lite slave arayüzü ve çevre birimleri SystemVerilog tabanlı UVM (*-ing* Universal Verification Methodology) ortamına entegre edilecektir.

UVM, çip tasarımlarının hatasız çalıştığını kapsamlı bir şekilde doğrulamak için SystemVerilog tabanlı olarak geliştirilmiş endüstri standardı bir test altyapısıdır. UVM, test ortamını birbirinden bağımsız ve tekrar kullanılabilir modüllere bölerek çalışır; temel olarak veriyi üreten, donanıma ileten, arayüzü dinleyen ve sonuçları bir referans modelle kıyaslayan birimlerden oluşur. Bu yapının temel kullanılma amacı; karmaşık donanımlarda insan eliyle tek tek yazılması imkânsız olan binlerce farklı test senaryosunu akıllı ve otomatik bir şekilde üreterek, sistemin beklenmedik durumlarda, uç vakalarda ve yoğun veri trafiğinde hata verip vermediğini yakalamaktır.

Bizim ticari EDA lisansı elde ettikten sonra amacımız ise, yarışma şartnamesinin beklentileri doğrultusunda, AXI4-Lite arayüzümüzü ve çevre birimlerimizi bu gelişmiş test ortamına entegre etmektir. UVM altyapısı sayesinde kendi tasarladığımız işlemcinin sonuçları ile Spike ISS referans simülatörünün çıktılarını otomatik olarak karşılaştıracak ve tasarımımızın her koşulda doğru çalıştığını endüstriyel standartlarda kanıtlamış olacağız.

GPIO: GPIO modülünün AXI4-Lite arayüzü üzerinden doğrulaması, Python tabanlı cocotb çerçevesiyle hazırlanan otomatik testler aracılığıyla gerçekleştirilmiştir. GPIO modülünün doğrulamasında saf Verilog yerine cocotb kullanılmasının temel nedeni, test sırasında bir AXI Master cihazını taklit etme zorunluluğudur. AXI4-Lite protokolünü Verilog'da karmaşık durum makineleri ile modellemek oldukça zahmetliyken, cocotb'nin Python tabanlı yapısı bu veriyolu işlemlerini esnek ve hatasız bir şekilde simüle etmemizi sağlamıştır.

Test süreci, modüle aktif-düşük (*-ing.* active-low) reset uygulanarak başlangıç durumuna getirilmesiyle başlamıştır. Ardından, AXI4-Lite el sıkışma (*-ing.* handshake) sinyalleri kullanılarak GPIO_ODR yazmacına 32-bit veri yazma işlemi gerçekleştirilmiş ve bu verinin donanımın fiziksel çıkış pinlerine (*-ing.* output_pins) sorunsuz bir şekilde yansıdığı doğrulanmıştır. Son aşamada ise, dışarıdan gelen sinyalleri temsil etmek amacıyla donanımın giriş pinlerine (*-ing.* input_pins) veri uygulanmış ve bu değerin GPIO_IDR yazmacı üzerinden sistem veriyolu aracılığıyla hatasız bir şekilde okunabildiği onanmıştır**.**

İlerleyen süreçte GPIO modülünün test kapsamı genişletilecektir. Bu doğrultuda UART üzerinden gelen veriye göre çeşitli GPIO çıktıları üretilecektir. Aynı zamanda GPIO giriş pini kullanılarak UART üzerinden mesaj gönderilmesi sağlanacaktır. GPIO testi kapsamında buzzer üzerinden kısa müzik parçaları çalınacak.

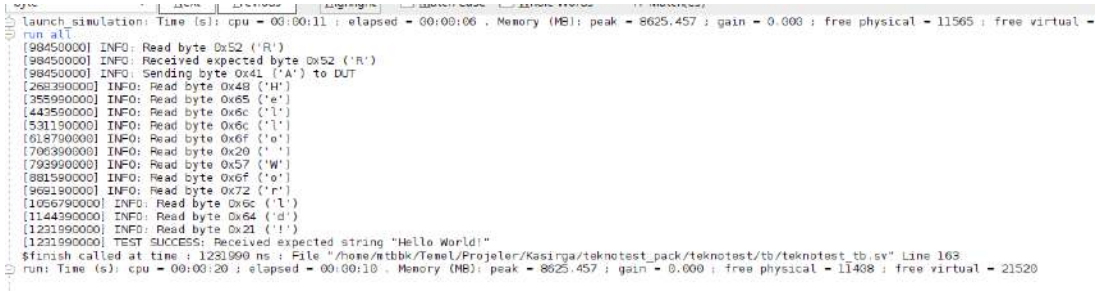
Timer: Timer modülünün sisteme entegrasyonu tamamlanmış ancak test çalışmalarımız devam etmektedir. Şu ana kadar yaptığımız test çalışmaları şu şekildedir: AXI4-Lite arayüzü olmayan timer modülü için Verilog testbench testi yazılmıştır. Bu testbench testinde timer modülünün temel işlevleri [prescaler, otomatik yeniden yükleme (*-ing.* auto-reload) değerine ulaşınca başa sarma, aşağı ve yukarı sayma, olay (*-ing*. event) yazmacı ve olay yazmacının sıfırlanması, sayaç değerinin sıfırlanması] testbench testindeki otomatik kontroller ve dalga formlarının incelenmesiyle gerçekleştirilmiştir. Timer modülüne AXI4-Lite arayüzünü ekleyen *timer_top* modülünün doğrulanmasında Python tabanlı cocotb çerçevesiyle hazırlanan 5 adet otomatik test gerçekleştirilmiştir. Bu testler kapsamında sırasıyla; yazmaçların reset sonrası varsayılan donanım değerleri doğrulanmış ve AXI4-Lite el sıkışma (*-ing.* handshake) sinyallerinin zamanlaması kanıtlanmıştır. Ardından, yapılandırma yazmaçlarına yönelik 32-bit tam sözcük okuma/yazma işlemleri ile WSTRB sinyali üzerinden yapılan kısmi bayt (*-ing.* byte-enable) erişimlerinin bütünlüğü başarıyla test edilmiştir. Son aşamada ise, zamanlayıcı aktifken donanım tarafından anlık güncellenen salt okunur sayaç (TIM_CNT) ve olay (TIM_EVN) yazmaçlarına sistem veriyolu üzerinden sorunsuz ulaşılabildiği doğrulanmıştır. Bu testlerde ilk olarak yazmaçların reset sonrası varsayılan donanım değerleri doğrulanmış ve AXI4-Lite el sıkışma sinyallerinin zamanlaması kanıtlanmıştır. Ardından, kontrol yazmaçlarına yönelik 32-bit tam sözcük okuma/yazma işlemleri ile WSTRB sinyali üzerinden yapılan kısmi bayt (*-ing.* byte-enable) erişimlerinin bütünlüğü başarıyla test edilmiştir. Son aşamada ise, zamanlayıcı aktifken donanım tarafından anlık güncellenen TIM_CNT ve TIM_EVN yazmaçlarına sistem veriyolu üzerinden sorunsuz ulaşılabildiği doğrulanmıştır.

İlerleyen süreçte Timer modülü için yapılan testlerin arttırılması için Timer modülü kullanılarak periyodik aralıklarla "." karakteri yazdırılacaktır. Bu işlem şu şekilde gerçekleştirilecektir: Öncelikle işlemcinin Timer ve UART modüllerini kullanarak periyodik olarak "." karakteri göndermesi için bir C programı yazılacaktır. Bu testin tamamlanmasından sonra Timer modülünün zamanlama hassasiyetinin doğruluğunu ölçme amacıyla PWM test kodları yazılacaktır. Bu kod farklı periyotlar ve çalışma çevrimlerinde (*-ing.* duty cycle) PWM sinyalleri üretecektir. Bu sinyaller, bir osiloskop vasıtasıyla gözlemlenerek modülün ne kadar hassas zamanlama yapabildiği analiz edilecektir.

UART: UART modülü için birkaç aşamalı bir geliştirme ve test süreci gerçekleştirilmiştir:

İlk olarak UART'ın iki ana alt modülü olan *uart_rx* ve *uart_tx* modülleri tasarlanmış ve bunlar üzerine AXI4-Lite arayüzü olmadan bir top modül yazılmıştır. Ardından bu top modülün testi öncelikle Vivado üzerinden Verilog testbench testi ile gerçekleştirilmiş ve modüllerin mantıksal olarak doğru çalıştığı teyit edilmiştir. Testbench testinin ardından modüllerin bit dosyası (*-ing.* bitstream) çıkartılmış ve FPGA içine aktarılmıştır. Bu aktarma işleminden sonra kişisel bilgisayarımızda çalıştırılan ve temel olarak bir USB portuna bit bit veri yollayan ve aynı porta gelen veriyi alarak terminale yazdıran bir C kodu çalıştırılarak FPGA içindeki UART modülümüzün testi gerçekleştirilmiştir. Raporun FPGA Prototipleme başlığı altında da bahsedildiği gibi bir FPGA entegrasyon testi ile modülümüzün uygulamalı olarak çalışır durumda olduğu belirlenmiştir. AXI4-Lite entegrasyonu olmayan UART modülümüzün testlerinin ardından *uart_top* modülümüze AXI4-Lite entegrasyonu yapılmış ve sıradaki test aşamasına geçilmiştir. Bu

aşamada AXI4–Lite entegre UART modülümüz eş zamanlı olarak yine takımımız tarafından hazırlanan ve raporun üst kısımlarında belirtildiği gibi işlemcimizin ana haberleşme protokolü olan OBI protokü ile ara bağlantı sağlayan OBI - AXI köprüsü ile bağlanmıştır. Ardından bu entegrasyon, işlemcimizin gerçekleştirmesini beklediğimiz eylemleri simüle etmek amacıyla tasarlanan bir Verilog testbench testi ile Vivado üzerinde test edilmiştir. Daha sonra AXI4-Lite entegre edilmemiş UART modülümüzde olduğu gibi bit dosyası oluşturulmuş ve AXI4-Lite entegre modülümüz yine kişisel bilgisayarda çalıştırılan aynı C kodu ile FPGA üzerinde test edilmiştir. Her iki FPGA testinde de başarı elde edilmiş ve raporun FPGA Prototipleme kısmında belirtilen çıktılar elde edilmiştir. Modülün tekil testlerinin ardından CV32E40P çekirdeği ile entegrasyonu sağlanmış; gerekli boot dosyaları, C kodları, Header dosyaları ve yardımcı, bağlayıcı modüller hazırlanmış ve işlemci entegre testlere geçilmiştir. Bu kapsamda kendi oluşturduğumuz testlerin yanı sıra yarışma komitesinin isterlerinden olan *dtr_demo* [30] Teknotest testi de Vivado 2021.2 sürümünde gerçekleştirilmiş ve başarı sağlanmıştır. Teknotest testinden elde ettiğimiz başarı sonucu Vivado terminaline yazdırılan sonuç Şekil 4.1.2'de görülmektedir.



Şekil 4.1.2: dtr_demo Teknotest Sonuç Çıktısı

Yapılan FPGA testi ve işlemci entegre testinin başarıyla sonuçlanmasının ardından, modülün güç tüketimi ve zamanlama davranışı gibi fiziksel özelliklerini değerlendirmek amacıyla ASIC gerçeklemesine yönelik çalışmalar yapılmıştır. Bu çalışmaların sonucunda sistemin gerçek üretim koşullarında nasıl davranacağının daha iyi anlaşılması hedeflenmektedir. Yarışma kapsamında kullanılacak ticari EDA araçlarına henüz erişim sağlanamadığı için, açık kaynaklı fiziksel tasarım akışı aracı olan OpenLane kullanılmıştır. Bu kapsamda UART modülümüz fiziksel tasarım akışından geçirilmiş ve OpenLane çip tasarım akışının bütün aşamalarından başarılı bir şekilde geçmiştir. Bu çalışma ile birlikte, tasarımın ASIC akışına uygunluğu test edilmiş ve ilk gerçeklemesi başarı ile tamamlanmıştır.

Modülün test kapsamı ilerleyen aşamalarda genişletilecektir. Ticari EDA araçlarına erişim sağlandığında daha kapsamlı entegrasyon çalışmaları, tasarımın daha detaylı doğrulanması ve iyileştirilmesi gerçekleştirilerek ASIC gerçeklemesinin nihai hale gelmesi sağlanacaktır.

I2C Master: I2C Master; *I2C_master.sv* ve *i2c_axi_wrapper.sv* adlı iki modülden oluşmaktadır. *I2C_master.sv* modülü I2C iletişim protokolü için gerekli olan sonlu durum makinesini (*-ing.* finite state machine) ve ayrıca senkronizasyon için gerekli olan saat (*-ing*. clock) sinyalini üreten modüldür. Bu modülü test etmek için SystemVerilog dilinde testbench testi yazılmış ve cocotb ile de test edilmiştir. cocotb ile test edilebilmesi için *cocotbext.i2c* kütüphanesindeki I2cMemory fonksiyonu ile somutlaştırılan (*-ing.* instantiate) golden slave ile *I2C_master.sv* modülünün uyumlu olabilmesi için

cocotb_for_golden_slave_wrapper.sv adlı bir wrapper yazılmıştır. Bunlara ek olarak donanım düzeyinde de test edilmiş olması amacıyla Vivado üzerinden *I2C_master.sv* modülüne ait bir verilog wrapper [Blok tasarımı (*-ing.* block design) oluşturulurken tasarıma eklenecek modüllerin Verilog dilinde yazılmış olması gerektiğinden dolayı], golden slave modülü ile Vivadonun VIO (Virtual Input/Output) ve ILA (Integrated Logic Analyzer) IP'leri kullanılarak bir blok tasarımı oluşturulmuştur. İşbu blok tasarım kullanılarak üretilen bit dosyası (*-ing*. bitstream) Basys3 FPGA kartına yüklenmiş ve yapılan test sonucunda oluşan "*.ila*" uzantılı dosya da ayrıca kaydedilmiştir. *i2c_axi_wrapper.sv* modülü ise I2C Master'ın sahip olduğu AXI4-Lite slave arayüzünü de bulunduran bir üst (*-ing.* top) modüldür. Bu modülün ilk testi cocotb ile yapılmıştır. Vivado üzerinde waveform ile simülasyon süreci ise devam etmektedir.

İlerleyen süreçte geliştirilen modülün I2C haberleşme kapasitesini doğrulamak amacıyla Basys3 FPGA kartı üzerinde donanımsal testler gerçekleştirilecektir. İletişimin sorunsuz sağlandığını teyit etmek için I2C protokolü ile çalışan MPU6050 [29] ivmeölçer ve jiroskop sensöründen veri okunacaktır. Üç eksende (x, y, z) açı ve ivme hesaplayıp bu değerleri dâhili yazmaçlarında depolayan bu sensör, 400 kHz I2C Master saat frekansını desteklemesi ve kapsamlı test olanakları sunması nedeniyle tercih edilmiştir. Uygulama aşamasında, test sürecinin zorlukları göz önüne alınarak ivme verileri kullanılmayacak; yalnızca sensörden I2C üzerinden alınan açı bilgileri, UART modülü aracılığıyla eşzamanlı olarak bilgisayara aktarılacaktır.

QSPI Master: QSPI Master modülü *qspi.sv*, *qspi_axi_wrapper.sv* ve *qspi_top.sv* modüllerinden oluşmaktadır. *qspi.sv* modülünün testi için SystemVerilog testbench testi yazılmıştır. Bu testbench testinde READ ve PP komutları x1, x2, x4 modlarında gönderilerek veri alışverişinin doğru çalıştığı gözlemlenmiştir. DOR komutu gönderilerek boş çevrim (*-ing.* dummy cycle) zamanlaması test edilmiştir. *qspi_axi_wrapper.sv* modülü *qspi.sv* modülüne AXI4-Lite arayüzü ve gerekli fifolarla olan bağlantıları sağlamıştır. *qspi_top.sv* modülü *qspi_axi_wrapper.sv* modülünün giriş çıkış hatlarını birleştirerek QSPI protokolünde olması gerektiği gibi tek hat hâline getirmiştir. *qspi_top.sv* modülüyle beraber AXI4-Lite arayüzünün testi için Python cocotb kütüphanesi kullanılarak bir test kodu yazılmıştır. Bu test kodunda QSPI Master modülünün fifolarla beraber veri alışverişi, farklı prescaler değerlerinde çalışması, okuma yazma işlemleri, fifo hata kodları durumları test edilerek QSPI Master modülü sisteme entegrasyona hazır hâle getirilmiştir. QSPI Master modülü için gereken sürücü (*-ing.* driver) C kodları yazılmıştır. QSPI Master modülü sisteme entegre edilmiştir. MT25QL256ABA8E12 flashının Verilog simülasyonu sisteme bağlanmıştır. İşlemci QSPI Master modülünü kullanarak flash belleğin cihaz kimliğini okumayı başarmıştır. Bu işlemle QSPI Master modülünün komut, adres göndermesi ve flashtan veri alma işlemlerinin doğru gerçekleştiği doğrulanmıştır. QSPI Master modülü işlemcinin önyükleme mekanizmasında kullanılmaktadır. Önyükleme işlemi sırasında işlemci QSPI Master modülünü kullanarak flash belleğe READ komutları göndermektedir. QSPI Master modülü başarılı bir şekilde bu komutları flash belleğe göndermiştir. Ardından flash belleğin READ komutu sonucu gönderdiği veriyi doğru bir şekilde RX fifoya yazmıştır. İşlemci QSPI_DR yazmacını okuyarak flash bellekten veri okuma işlemini başarılı bir şekilde tamamlamıştır. Bu testlerle QSPI Master modülünün şartnamedeki isterlere uygun çalıştığı doğrulanmıştır.

Yapay Zeka Hızlandırıcı:

Yazılan hızlandırıcı modüllerini test etmek için testbench testleri yazılmış ve Vivado üzerinde simülasyon aracılığıyla doğru çalıştıkları doğrulanmıştır. Mevcut testbench testleri şu şekildedir:

1. ***processin_element_tb.sv***: Yan yana bağlı olacak şekilde iki processing_element modülü gerçekler ve belli çevrimlerde modüllerin girişlerine değerler gönderir. Çarpma, biriktirme ve modüller arası 1 çevrim gecikmeli veri aktarımının beklenen şekilde çalıştığını doğrular.
2. ***wrapper_tb.sv***: İçerisinde *stagger* ve *systolic_array* modüllerini gerçekleyen *sa_wrapper* modülünü test eder. Testbench içerisinde tanımlı iki adet matrisi *sa_wrapper* modülüne besleyerek matris çarpımı yapmasını sağlar. İşlem yapıcı birimlerdeki sonuçlar çarpım sonucuna uyuyor mu kontrol eder. Hatalı sonuç varsa hangi *processing_element* modülünde olduğunu yazar yoksa "hepsi doğru" çıktısıyla wrapper modülünün, içerisindeki stagger ve sistolik dizi modülleriyle birlikte doğru şekilde matris çarpımını yapabildiğini doğrular.
3. ***feedertb.sv***: *im2col_feeder* ve arabellek modülü bağlanıp çalıştırılmıştır. Mevcut test formatı için arabellek modülü içerisinde başlangıç bloğunda *convdenememetrisi14x46.mem* dosyasından okunan bir matris bulunmaktadır. Simülasyon esnasında dalga formu incelenerek matris verilerinin im2col işlemine uygun şekilde i modülünden çıktığı doğrulanmıştır.
4. ***accelerator_top_tb***: İçerisinde *im2col_feeder* modülünü buna bağlı sa_wrappe modülünü evrişim işlemini yapabiliyor mu diye kontrol eder. Hata çıkması halinde sonucun çıktı arabelleğine yazıldığı adresi, beklenen ve hesaplanmış sonucu yazar. Hata yoksa başarılı şekilde çalıştığını yazar.

Tasarımın devam edecek sürecinde uygulanacak test planı şu şekildedir.

1. Hızlandırıcı modüllerinin tamamı tamamlandıktan sonra modülleri teker teker test etmek için testbench testleri yazılacak ve simülasyon ile doğrulamaları yapılacaktır.
2. Sistem bir araya getirildikten sonra modelin yazılım gerçeklemesiyle kıyaslanarak RTL tasarımının doğru sonuçları üretebildiği test edilecektir.
3. FPGA üzerinde RTL gerçeklemesi çalıştırılarak testlere devam edilecek, bu aşamada kritik yol analizleri ve FPGA üzerinde çalıştırılmasına yönelik geliştirme ve düzeltmeler yapılacaktır.

5. Takım Organizasyonu

5.1. Takım Tanımı



Proje takımımız Kasırga - Ay'ın bünyesinde 5 tane lisans öğrencisi ve 1 danışman bulunmaktadır. Lisans öğrencilerinin her biri TOBB ETÜ Bilgisayar Mühendisliği bölümünde eğitimlerine devam etmektedir. Hazırlanan

takım şemamız Şekil 5.1.1'de görülmektedir.

Şekil 5.1.1: Takım Şeması

5.2. Görev Dağılımı

ÖTR aşamasında proje 7 iş paketine bölünmüştür. İşbu paketler: Gereksinimlerin Belirlenmesi ve ÖTR Hazırlanması, Çevre Birimlerinin Tasarlanması, Yapay Zeka Hızlandırıcının Tasarımı, Sistem Entegrasyonu, Doğrulama, DTR Hazırlanması, Tasarımın Nihai Hale Getirilmesi ve Final Sunumu Hazırlığı şeklindedir.

İlk iş paketi olan Gereksinimlerin Belirlenmesi ve ÖTR Hazırlanması projenin temelini oluşturduğu için takımın her üyesi bu aşamada rol almıştır ve bu paket tamamlanmıştır. Çevre Birimlerinin Tasarlanması ÖTR'de de belirtildiği şekilde takımdaki üyelere paylaştırılmıştır. Timer ve QSPI Master Osman, I2C Master Taha, UART Barbaros ve GPIO ise Selim tarafından tasarlanmıştır. Bu iş paketi de takımımızca tamamlanmıştır ancak ilgili çevre birimlerinden I2C Master, GPIO ve Timer'ın testleri devam etmektedir. Planlamaya uygun ilerlememiz ve iş paketlerinde gecikme yaşanmaması hâlinde bir JTAG modülünün tasarlanması ve sisteme entegre edilmesi planlanmaktadır. Yapay Zeka Hızlandırıcının Tasarlanması iş paketi devam etmektedir, Alperen bu aşamada rol almaktadırlar. Sistem Entegrasyonu süreci de hâli hazırda devam etmektedir. Şimdilik UART ve QSPI Master modülleri testleriyle beraber sisteme entegre edilmiş kalan modüllerin testleri devam etmektedir. Bu kısımdan Barbaros ve Osman sorumludur. Bu iş paketinin bir başka aşaması olan C sürücüleri ile AXI4-Lite arayüzleri üzerinde ise Selim çalışmaktadır. Doğrulama süreci daha önce belirtildiği üzere devam etmektedir. SystemVerilog testleri ve cocotb testleri ile birçok farklı senaryo test edilmiştir. Daha fazla testin yapılabilmesi için UVM ortamı hazırlanmaya başlanacaktır. Sistemin bir bütün olarak test edilmesi tüm çevre birimlerinin tekil sistem entegrasyonu testleri tamamlandıktan sonra gerçekleştirilecektir. Bu iş paketinden bütün takım sorumludur. Tasarımın Nihai Hâle Getirilmesi ve Final Sunumu Hazırlığı olan son iş paketine ise şimdilik başlanmamıştır ve bu aşamada da tüm takımın rol alması planlanmaktadır.

6. İş Planı ve Risk Planlaması



Şekil 6.1.1: İş Planı

Henüz tamamlanmamış ve devam etmekte veya henüz başlanmamış işlerin planlaması Şekil 6.1.1'de görülmektedir. Proje şartnamesinde JTAG modülünün zorunlu olmadığı belirtilmiştir, bu nedenle Yapay

Zeka Hızlandırıcı aşamasında meydana gelebilecek olan herhangi bir aksaklık sebebiyle JTAG modülünün tasarlanması ertelenebilir ya da iptal edilebilir durumdadır. Çekirdeğin doğrulanması işi Test başlığı altında da belirtilen Coremark testi ile çekirdeğin test sayısının arttırılmasını kapsamaktadır. Bu iş de benzer şekilde şartnamede istenmemiştir ancak proje sürecinin planlandığı gibi ilerlemesi durumunda test sayısını arttırmak amacıyla takımımızca yapılmasına karar verilmiştir. İş planında yarışma ile ilgili işlerin bitirilmesi için planlanan tarih 13 Temmuz Pazartesi olarak belirtilmiştir. Yarışma takviminde Tasarımın Nihai Hâle Getirilmesi olarak belirtilen 31 Temmuz Cuma günü ile arasındaki 18 günün ise sistemin başarımını iyileştirmek geliştirmeler ve ek testler yapmak için kullanılması planlanmaktadır. Ayrıca bir aksaklık yaşanması durumunda da bu süre kullanılacaktır.

Olası Risk Durumu	Etki Derecesi	Olasılık
Entegrasyon Testlerinin Tamamlanamaması	5	1
Yapay Zeka Hızlandırıcının Yetiştirilememesi	5	2
JTAG Modülünün Yazılamaması	2	3
FPGA Prototiplemenin Yetiştirilememesi	4	2

Etki Derecesi
(1) Çok Hafif (3) Orta Derece
(2) Hafif (4) Ciddi
(5) Çok Ciddi Olasılık

Olasılık
(1) Çok Az (3) Orta Derece
(2) Az (4) Yüksek
(5) Çok Yüksek

Şekil 6.1.2: Risk Tablosu

Mikrodenetleyiciyi tasarlarken yaşanabilecek sıkıntılara dair risk verileri, bu risklerin etki dereceleri ve bu risklerin yaşanma olasılıkları Şekil 6.1.2'de tablo hâlinde görülmektedir. Risk tablosundaki etki derecesi, olasılık değişkenleri en yüksek 5, en düşük 1 olacak şekilde ölçeklenmiş ve bu ölçeğe uygun şekilde risk ataması yapılmıştır.

7. Kaynakça

[1] OpenHW Group, “CV32E40P.” [Online]. Available:

https://github.com/openhwgroup/cv32e40p

[2] https://github.com/MTBBK/2026_teknofest_kasirga-ay

[3] RISC-V Software Source, riscv-isa-sim (Spike) GitHub, [Online]. Available: https://github.com/riscv-software-src/riscv-isa-sim

[4] https://github.com/MTBBK/2026_teknofest_kasirga-ay/blob/main/Proje/Test/spike_test/tb_top.sv

[5] Xilinx, "Vivado ML Overview," [Online]. Available:

https://amd.com/en/products/software/adaptive-socs-and-fpgas/vivado.html

[6] “cocotb,” 2025. [Online]. Available:

https://github.com/cocotb/cocotb

[7] BASYS 3 FGPA, [Online]. Available:

https://digilent.com/reference/programmable-logic/basys-3/start

[8] The OpenROAD Project, OpenLane. [Online]. Available: https://github.com/The-OpenROAD-Project/openlane

[9] https://github.com/MTBBK/2026_teknofest_kasirga-ay/tree/main/Proje/OpenLane

[10] "RISCV GNU Toolchain," GitHub, [Online]. Available: https://github.com/riscv-collab/riscv-gnu-toolchain

[11] https://github.com/MTBBK/2026_teknofest_kasirga-ay/blob/main/Proje/Boot/uart.h

[12] Micron, “256Mb, 3V Multiple I/O Serial NOR Flash Memory Datasheet”, [Online]. Available: https://mm.digikey.com/Volume0/opasdata/d220001/medias/docus/6657/MT25QL256ABA8ESF-0SIT.pdf

[13] https://github.com/MTBBK/2026_teknofest_kasirga-ay/tree/main/Proje/Accelerator

[14] TensorFlow, “micro_speech Github Repository” [Online]. Available: https://github.com/tensorflow/tflite-micro/tree/main/tensorflow/lite/micro/examples/micro_speech

[15] TensorFlow, "micro_speech_quantized.png," *GitHub Repository: tflite-micro*, 2026. [Online]. Available: https://github.com/tensorflow/tflite-micro/blob/main/tensorflow/lite/micro/examples/micro_speech/images/micro_speech_quantized.png (accessed: Jun. 15, 2026).

[16] D. Podareanu, V. Codreanu, and S. Aigner, "Best Practice Guide - Deep Learning," C. van Leeuwen and V. Weinberg, Eds., Partnership for Advanced Computing in Europe (PRACE), Version 1.0, Feb. 2019.

[Online]. Available: https://prace-ri.eu/wp-content/uploads/Best-Practice-Guide_Deep-Learning.pdf (accessed: Jun. 15, 2026).

[17] L. Roeder, “Netron,” GitHub Repository. [Online]. Available:

https://github.com/lutzroeder/netron

[18] Google AI Edge Team, “Model Explorer,” GitHub Repository. [Online]. Available: https://github.com/google-ai-edge/model-explorer

[19] Y. Xiao, Z. Chen, and Z. Zhong, "FPGA Term Project - Simple TPU," *GitHub Repository: FPGA-Project-2022-simple-tpu*, 2026. [Online]. Available: https://github.com/yuyuranium/FPGA-Project-2022-simple-tpu/blob/main/README.md (accessed: Jun. 15, 2026).

[20] J. Fornt Mas, M. Caro Roca, J. Abella Ferrer, F. de Borja Moll Echeto, and J. Altet Sanahujes, "An Energy-Efficient GeMM-Based Convolution Accelerator With On-the-Fly im2col," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 31, no. 11, pp. 1874-1878, Nov. 2023, doi: 10.1109/TVLSI.2023.3286122.

[21] https://github.com/MTBBK/2026_teknofest_kasirga-ay/tree/main/Proje/Boot

[22] https://github.com/MTBBK/2026_teknofest_kasirga-ay/blob/main/Proje/PeripheralModules/UART/test/pc_uart_test.c

[23]. S. Chen, B. Shi, and Y. Yuan, “Gradient Norm Minimization of Nesterov Acceleration: o(1/k^3),” arXiv:2209.08862, Sep. 2022. [Online]. Available:

https://arxiv.org/abs/2209.08862

[24] Wikipedia Contributors, “A* Search Algorithm,” Wikipedia, The Free Encyclopedia. [Online]. Available: https://en.wikipedia.org/wiki/A*_search_algorithm

[25] F. Rubin, “The Lee Path Connection Algorithm,” IEEE Transactions on Computers, vol. C-23, no. 9, pp. 907–914, Sep. 1974, doi:

https://ieeexplore.ieee.org/document/1672647

[26] https://github.com/MTBBK/2026_teknofest_kasirga-ay/blob/main/Proje/Test/spike_test/Test_code/test.c

[27] D. Gisselquist, “AXI-Lite Formal Verification Properties (faxil_slave.v),” GitHub Repository: wb2axip. [Online]. Available: https://github.com/ZipCPU/wb2axip/blob/master/bench/formal/faxil_slave.v

[28]YosysHQ, “SymbiYosys (SBY),” GitHub Repository. [Online]. Available: https://github.com/YosysHQ/sby

[29] SparkFun, “MPU-6000 and MPU-6050 Product Specification,” [Online]. Available: https://cdn.sparkfun.com/datasheets/Components/General%20IC/PS-MPU-6000A.pdf

[30] Teknotest dtr_demo Available: https://github.com/MTBBK/2026_teknofest_kasirga-ay/tree/main/Proje/Test/dtr_demo